Lex’it HowTo

Version 2026.10.20

IvdNT

Mathieu Fannee, 2019 – 2026

Inhoud

Introduction 9
What is Lex'it? 9
API 9
Installation: quick start 10
Prerequisites 10
Build with MAVEN 10
Copy needed files to the server 11
Set the access control database 12
Access the admin GUI 13
My first project 15
The .database file 16
The .config.js file 16
Remarks 17
Call your Lex'it project via a URL 19
Full list of configuration parameters 20
oTableSettingsList parameters 20
oTableConfigurationList parameters 25
Giving access to others 32
Assign rights to users 32
Setting rights 32
Show your projects in a menu 34
Use the projects_overview.js file 34
Use the Admin GUI 35
The API 37
First things first: open a table 37
Close a table 38
Project-wide settings 39
Setting a project title, background-color, font, etc. 39
Setting table group in table menu 40
Table global settings 41
General settings 41
Is a table loaded? 41
Get the name of a table 41
View type 41
Row selection 42

Is a table empty? 42
Table results count quality 42
Positioning 42
Table absolute positioning 42
Table relative positioning 43
Saving and retrieving settings 44
Read the configuration settings 44
Restore the configuration settings 44
Save table extra settings (position etc.) for later re-use 44
Save and restore the table state 45
Respawn a table 46
Retrieve the table wrapper - Access table HTML elements 46
Table header 47
Ways to set header buttons 47
Custom buttons 47
Styling a button 47
Add a SELECT button 48
Implement a button for row duplication 48
Click on a button programmatically 49
Modify the name/style of a button programmatically 49
Free buttons 50
Filters & search boxes in the GUI 52
Data types, operators, etc. 52
Logical operators 53
Searchboxes 54
Modify a user's search query programmatically, before search is fired 55
Modifying the filters silently (that is: without modifying the search boxes) 56
Change the search values of a dropdown searchbox 57
Query builder 58
Table-body 59
Some basic cell configuration 59
Set a wysiwyg editor onto a cell 60
Dealing with rows 61
Get the first row 61
Which row is currently active? 61
Get the row before/after another row 61

Which row is now selected? At which index? 61
How many rows are now selected? 61
How many rows are now visible on screen? 61
Iterating over rows 62
Get number of rows 62
Get the index of a row (row-number) 62
Get all visible rows 62
Get all visible rows meeting some filters 63
Select or unselect row programmatically 63
Refresh the row data 63
Dealing with cells (columns) 64
Get a cell and its type, given a row node and a column name. 64
Check if a column is visible, etc. 64
Assign a mouse event to a cell 64
Assign a mouse event to an element in a cell 65
Highlighting data in cells 66
Sorting data 66
Read data 68
Read text from cells, columns, or rows 68
Read selected text in cell 69
Read IDs of rows 69
Dealing with Booleans 70
Write data 71
Write data in editable cells 71
Sending data to the database 72
Context menus in cells or rows 74
Refreshing a table, or only single rows 75
Remove data 75
Remove a row 75
Implement an autocomplete in an editable cell 76
Wildcards in columns configuration 77
Navigate programmatically 78
Table footer 79
Pagination buttons 79
Compact footer 79
Export buttons visibility 79

Add a full export button 80
Callbacks 81
Table callbacks 81
Function callbacks 83
Error handlers 84
Table columns error handlers 84
Function error handlers 84
Show a clean error message, when the database threw an exception 85
Dialogs 86
Classic dialogs 86
Entering data in a dialog 86
Data to be typed in 86
Data to be chosen out of a list of items 87
Data to be sorted 88
Menu dialog 88
Spread data among tabs in dialog 89
Get rid of the 'OK' button 90
Repositioning a dialog 90
Closing a dialog programmatically 90
Show a progress indicator and hide it 91
Key events 92
Assign event to some key 92
Trigger a key event programmatically 92
Detect which key was struck 93
Calling external resources: 94
Call a database function and get its output 94
Call a webservice 95
Proxy and crossDomain 95
Form views 96
Basic configuration 96
Putting cells into groups 99
Callbacks 100
Tables as part of a form: lists 101
Basis configuration 101
The 'add' button 104
The 'delete' button 106

The ‘show’ button 107
Overview of the form API 108
Forms – retrieve configuration 108
Forms - Read, write and refresh data 109
Forms - search boxes 111
Lists – retrieve configuration 112
Lists - Read, write and refresh data 113
Lists – dealing with cells 114
Lists – dealing with rows 115
Lists – dealing with columns 117
Tips and tricks 118
Lex’it internal registry 118
Using wildcards in configuration 119
Updating a table configuration after initialization 120
Create a full new table configuration at runtime 121
Set the active table programmatically 121
Use libraries 122
Use multiple config.js files 123
Load new CSS dynamically 124
Accessing session and user info 125
Cleaning the cache 125
Setting behavior when switching between tabs containing distinct Lex’it instances 126
Changing accessed table schema 126
Custom sorting 127
Create a Welcome or Login page 129
Call a Lex’it projects and tables via a URL 130
Set the language of the GUI 131
Change table columns visibility and redraw table automatically 131
Scroll to a table programmatically 131
Synchronicity problems and solutions 132
Implement a radio button using Boolean columns 133
Implement a character picker 134
Allow users in without logging in 135
Help functions 136
String array conversion 136
Escaping chars 136

Http parameters 136
Solve z-Index problems 136
Objects functions 137
Clone an object 137
Count the number of properties of an object 137
Regex functions 137
Test if a string is a regex or not 137
Regex version of replaceAll(string, replacement) 137
Regex version of indexOf(regex, from) and lastindexOf(regex, from) 137
Regex version of inArray(value, array, start) 137
Regex selectors 137
Arrays functions 137
Unify any value in array 137
Clone an array 138
Array equality 138
Sort an array of arrays 138
HTML entities 138
Numbers 138
Generate series (JavaScript version of PostgreSQL function) 138
Get a random unique number 138
String functions 139
The good old BASIC or JAVA functions 139
String replacement 139
Check for HTML tags, and remove those from a string 139
Remove non alphabetical chars from string 139
If a string a string? 139
If a string truly empty? 139
Colors 139
Is a color dark or light? 139
Convert a string to a color 140
Make color a bit lighter or darker 140
Convert HEX to RGB color code, or back 140
Text selection 140
Allow or prevent screen text selection 140
Clear screen text selection 140
DOM elements functions 140

Does an element exist? 140
Test if some element is visible in the current viewport 140
Events 141
Pre-binding 141
PSQL functions in Lex’it 142
Call a PSQL-function and read the result 142
Concordance table for the fn and fx namespaces 143
Concordance table for forms vs. other namespaces 146
Concordance table for lists vs. other namespaces 146

Introduction

What is Lex’it?

In short: a customizable web interface on databases. Out of the box, it allows for viewing or editing database tables within a webpage, and offers searching, paging and sorting functionalities without any configuration needed.
Thanks to an extensive API, developers can easily customize or extend the standard functionalities. And programming is not limited to the Lex’it API: since it all runs with JavaScript and jQuery, lots of third parties JavaScript plugins can be implemented in Lex’it projects.

API

The client-side part of Lex’it is built with the dataTables.net framework, so as a consequence the Lex’it API uses the same objects as DataTables.

Since its creation, the DataTables API (https://datatables.net/reference/api/) has gone through a lot of development, causing its API to shift from one type of objects to another. Originally addressing the DOM tree and its nodes, DataTables now works with its own table representation object. The Lex’it API still uses both approaches: the first one is represented by the `fn` namespace (mostly addressing nodes) and the other by the `fx` namespace (mostly addressing DataTables objects).

Keeping both approaches has a clear advantage: since `fn` namespace functions primarily address nodes, those functions give direct access to elements to be styled with CSS, etc. The `fx` namespace functions, on the other hand, allow us to keep up with the current DataTables API. The Lex’it API of course provides a set of functions allowing both function types to be used together.

The Lex’it API is described at `github.com/instituutnederlandsetaal/Lexit/tree/master/jsdoc`, so we won’t be doing that again here. But do consult the **Concordance table for the `fn` and `fx` namespaces** in this document!

The aim of this document is to give clear examples of how to use some functions. This tries to give an answer to the question: how can I get Lex’it to do this for me?

Installation: quick start

Prerequisites

- A server running Java 11+ and Apache Tomcat 10.1+
  which is needed to host the Lex’it webservice
- A server (might be the same as the first one) running PostgreSQL 10+
  containing the database to be viewed or edited in Lex’it

Build with MAVEN

Lex’it is stored in GitHub : https://github.com/instituutnederlandsetaal/Lexit

Clone it and build it:

```
git clone https://github.com/instituutnederlandsetaal/Lexit

cd Lexit
mvn package
```

You now have a WAR file (`lexit2.war`) ready to be installed on the Tomcat server.

Copy needed files to the server

We need some files to be copied to the Tomcat server, that is:

- `lexit2.war` (the WAR file you just built)
- the `/lexit/lexit2_config_folders` folder (projects configuration folders)

Send those to the Tomcat server:

```
scp lexit2.war root@<your_host>:<some_folder>
scp -r /lexit2_config_folders root@<your_host>:<some_folder>
```

The 'lexit2_config_folders' folder contains two subfolders:

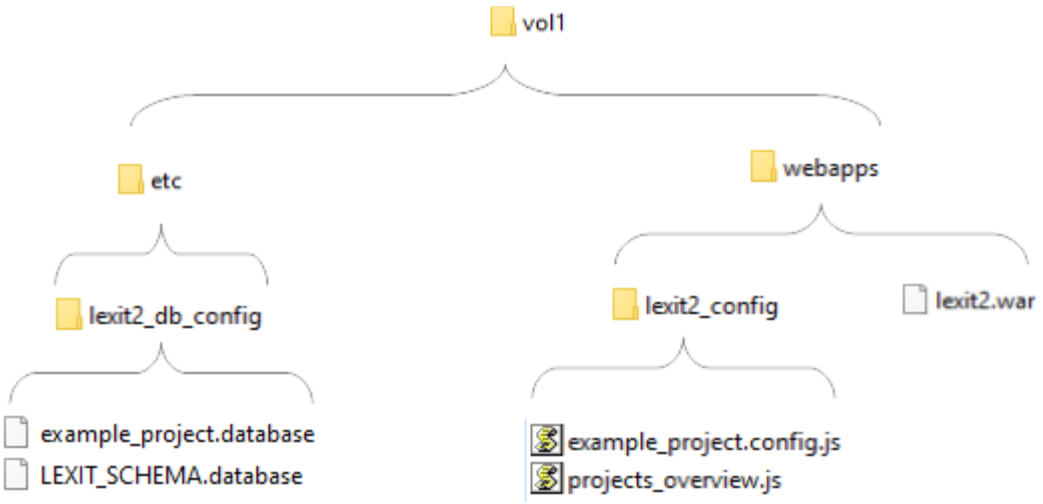
- `/webapps/lexit2_config`
  This folder is about projects functionalities. It is supposed to contain JavaScript files only!
- `/etc/lexit2_db_config`
  This folder contains the projects' databases credentials, and also the credentials of the Lex'it access control database.

Now go to the Tomcat server, copy the war file and the `lexit2_config` folder to the Tomcat webapps folder (e.g. `/vol1/webapps`), and copy the `lexit2_db_config` folder to the parent of the webapps folder (e.g. `/vol1`).

```
ssh root@<your_host>

mv lexit2.war /vol1/webapps
mv lexit2_config_folders/webapps/lexit2_config/ /vol1/webapps/
mv lexit2_config_folders/etc /vol1
```

You should end up with the following structure:



Note: In the older Lex’it versions, all config files were stored in the `/webapps/lexit2_config` folder, but it turned out to be a bad idea. Since the `webapps` folder can be accessed by the client, it can form a security risk [remember: it contains the database credentials]. That’s why we eventually chose a new location, which can’t be accessed by the client.

Set the access control database

In Lex’it, usernames, passwords and roles are kept in a PostgreSQL database. We call this access control database the ‘Lex’it access schema’. The location and credentials of this database are declared in the `LEXIT_SCHEMA.database` file mentioned hereabove, as part of the `/etc/lexit2_db_config` folder. Its content is this:

```
#
#
# WARNING: The Lex'it schema is an access control database, which means
# that it holds the usernames, passwords and access roles of all Lex'it users.
#
# Be very cautious: Leave the management of this database to Lex'it.
# Use the admin GUI to add users and define their roles. The admin GUI can be
# accessed at:
#
#      http://<your_host>:8080/lexit2/?db=admin
#
# Note: the recommended default database name is LEXIT_SCHEMA, but you might want
# to call it otherwise (see: value of db parameter).
#
db=LEXIT_SCHEMA
schema=public
host=yourhost.com
user=username
pass=password
```

Edit this file to declare the hostname of your PostgreSQL engine, and the username and password required to access it. As a final step, you’ll need to create that database.

```
psql –h yourhost.com –U username –d postgres –c 'CREATE DATABASE "LEXIT_SCHEMA";'
```

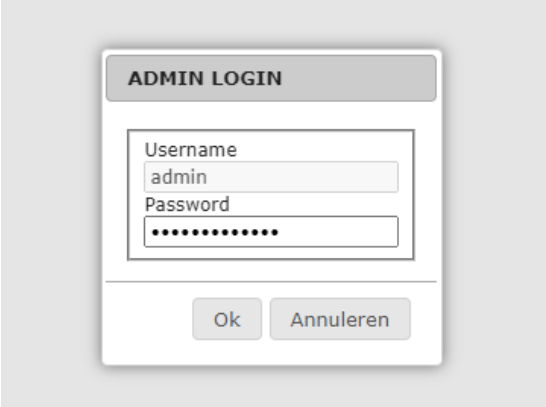
You won’t need to add any content to it: Lex’it does that automatically, as soon as you try to access it. We’ll do that in the very next section.

Access the admin GUI

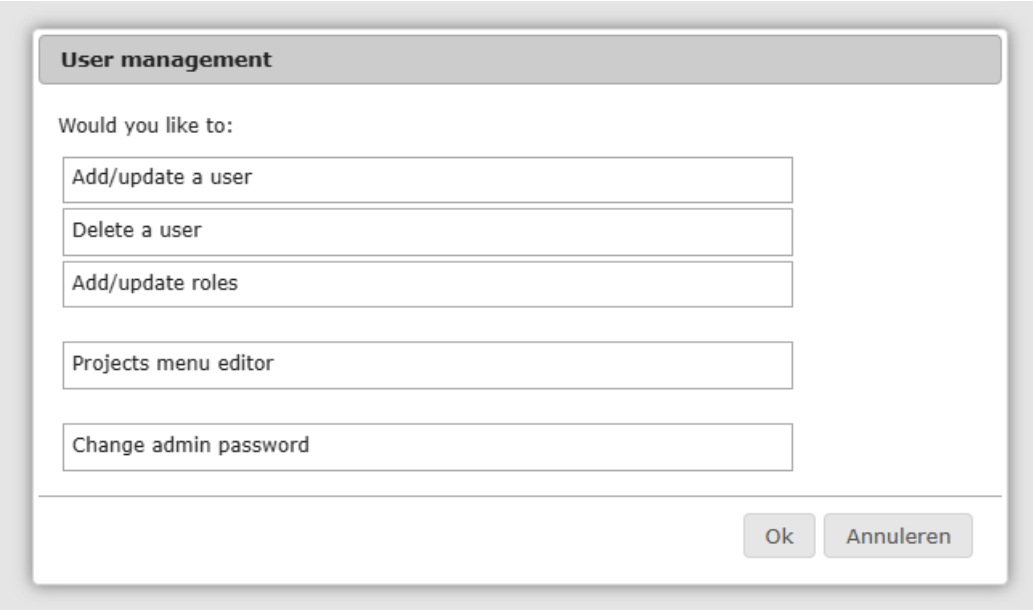
Go to the Lex'it admin screen by typing this URL:

```
http://<your_host>:8080/lexit2/?db=admin
```

That will open the admin login dialog:



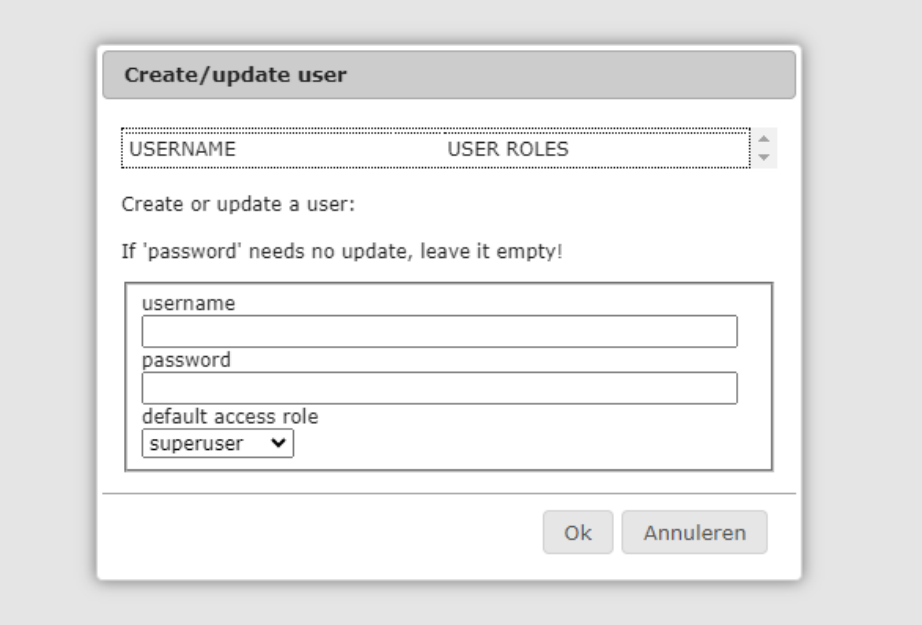
Log in with the admin password (default: `KrommeBananen`), and the following screen will open:



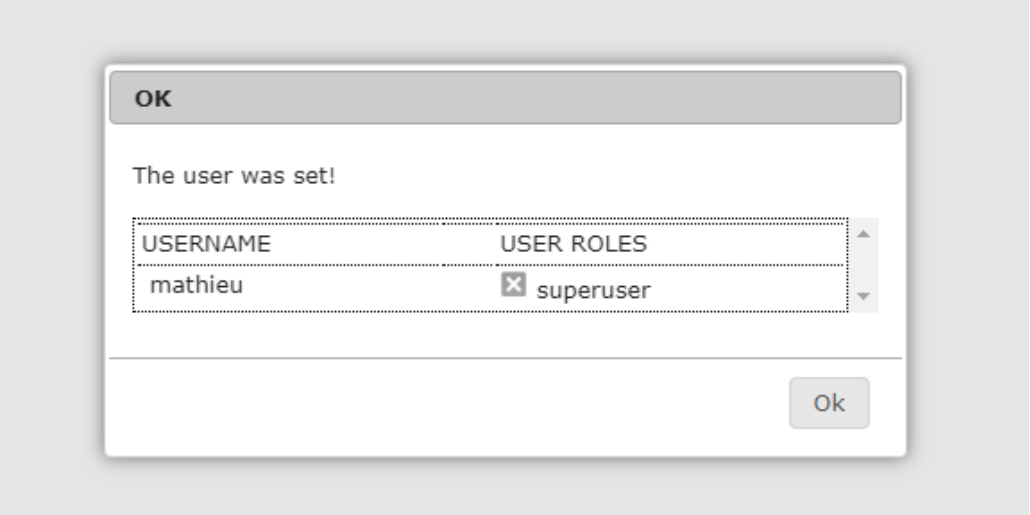
The first thing to do when accessing the admin section for the first time is **change the admin password**. We strongly advise to do it, for obvious security reasons. To do so, just click the 'Change admin password' option, enter the current password and a new password. Done!

As far as the other options of the menu are concerned, those are quite self-explanatory: you can add users or delete them, and you can assign them particular access roles.

Obviously, you will need to declare yourself as a user. Just type in a username and a password. For the sake of ease, assign yourself the 'superuser' role (unlimited access) and click OK.



If all goes well, you should get a confirmation dialog like this. This shows that a user with username 'mathieu' was created, and it was assigned the 'superuser' role:



Now, you have a user account and unlimited access. In the next section, you're going to create your first project.

My first project

A project needs only two files:

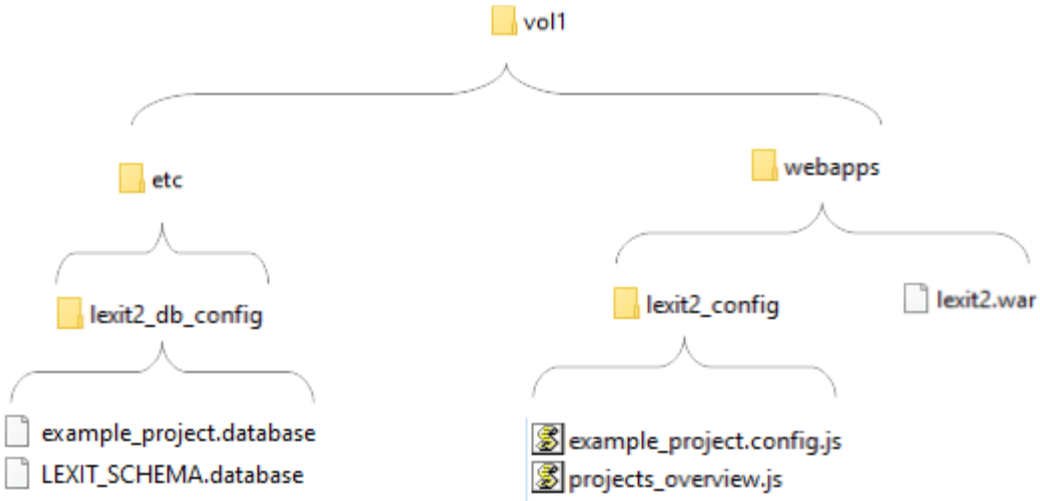
- a `.database` file, which contains the project database location and credentials
- a `.config.js` file, which indicates which database tables can be accessed, how they should look and behave, etc.

These two files need to be named the following way:
if your project is named 'myproject', those two files should be called `myproject.database` and `myproject.config.js` respectively.

As for their storage location:

- The `myproject.config.js` file must be stored in the Tomcat webapps folder in 'configuration' subfolder `/webapps/lexit2_config`
- The `myproject.database` file must be saved at `/etc/lexit2_db_config`

As we saw before (in the **Installation: quick start** section), the `etc` folder was created next to (that is: as a sibling of) the `webapps` folder:



The .database file

A typical `.database` file looks like this:

```
db=my_project_database
schema=public
host=somehost.com
user=postgres
pass=password
port=5342
max_pool_size=10
collation=nl_NL.utf8
send_username_to_db=true
```

Most of the content is straightforward:

- **db** is assigned the name of the project database.
- **schema** is assigned the database schema Lex'it is allowed to access in the project database (note that this allows you to easily keep parts of a database hidden, since Lex'it will only be able to access the specified schema; but also note that the targeted schema can be changed at runtime with the `fn.setSchema` function, if you need that to happen in a project).
- **user** and **pass** are of course needed for Lex'it to access the database.

The following parameters are optional:

- **port** is the TCP port on which the server is listening for connections. If you use the default port 5342, this parameter can be left out.
- **max_pool_size** is about the maximum number of connections to be held in the connection pool of a project. It has a default value of 10. But if you expect many users to access your project at the same time, you may choose a higher value. **Be aware** that PostgreSql has a 'max_connections' parameter too (to be found in postgresql.conf), which has value 100 by default.
- **collation** can be used to change the default sorting order, by specifying a collation to apply. In PostgreSql queries this will affect any clause or operator needing a collation to operate (e.g. `<, >, ORDER BY`).
- **send_username_to_db**[1] is needed when you want to be able to identify (and store) the users' identity for logging purposes etc. (see the **Accessing session and user info** section).

The .config.js file

A default `.config.js` file looks like this:

```
oHiddenTablesList = [];
oShowOnlyTables = [];

oTableSettingsList = {};
oTableConfigurationList = {}
```

[1] In older Lex'it versions, which worked with Tomcat login, this parameter was called 'send_tomcat_username_to_db'. For backwards compatibility, this parameter still exists, but it now contains the Clarin or Lex'it username, instead of the Tomcat username as in the past.

The config file consists of four arrays. The first two arrays are about tables' visibility. One array must be used to declare the names of the tables that must keep hidden, while the other is about tables that must be visible. Those arrays are mutually exclusive: only one can be used at a time. If only a few tables need to be visible, and lots of tables must keep hidden, using **oShowOnlyTables** makes more sense as only a few names will have to be declared. If, on the contrary, many tables must be visible, and only a few must be hidden, you should use **oHiddenTablesList** instead.

The two following arrays are associative arrays:

- The **oTableSettingsList** array is used for table settings like: table width, header color and buttons, etc.
- The **oTableConfigurationList** array is to be used for configuration at column level, like click events assigned to cells, cells editability or visibility, etc.

These arrays are associative arrays, so as to be able to assign a set of settings to a table name (*top-level key*) and column names (*inner object keys*). The table or column names will act as parameter names (or keys), and the setting will act as a parameter value.

Let's give a short example. Let's say we have a table called 'my_table', with only two columns: a 'word' and its 'ID'. With only two columns our table is quite narrow, so we set it to occupy only 50% of the available screen width. And we want to sort the table by its ID column in ascending order. Those *table-wide* settings are to be declared in the **oTableSettingsList** array.

The second array – **oTableConfigurationList** – is about configuration *at column level*. Here is the place where you can specify how a column should render or behave. In this particular case, we want the ID column to be hidden. And we want the column 'word' to be editable, meaning that the user will be able to click on any cell of this column to edit its value. To achieve all that, the configuration file should look like the following:

```
oTableSettingsList = {

	"my_table": {
		"width": "50%",
		"columns_sorting": {"id": "asc"}
	}
};

oTableConfigurationList = {

	"my_table": {
		"id": {
			"visible": false
		},
		"word": {
			"editable": true
		}
	}
};
```

Remarks

It may seem easy — but be careful.

Although Lex'it is aiming at making everything as simple as possible (including all kinds of database querying optimizations), there remain things that we need to take care of ourselves. In this case: if a

table needs to be editable, in the database it will need a primary key (which gives Lex'it a way of pointing to the table row to be edited). Typically, the primary key is the ID column.
So all you'll need to do in your database is setting a primary key for the table to be edited, and Lex'it will be able to use it without any further configuration:

```
ALTER TABLE my_table ADD primary key( id );
```

Editing a database view is possible as well, provided that the view has rules declared allowing insert, update and/or delete operations. Since a database view doesn't have such thing as a primary key (needed for table edition, see above), we need to use a trick. Choose a column that can act as a primary key (that is: it contains only unique values) and give it '`pkid`' as an alias in the view declaration. Lex'it will recognize it automatically and use that column as if it was a genuine primary key on a table:

```
CREATE OR REPLACE VIEW my_view
 AS
 SELECT column_1 AS pkid, column_2, column_3
 FROM my_table;
```

> **Important note**: in its current version, Lex'it does not yet support primary keys on multiple columns (such composite primary keys are rare anyway).

But other programming strategies are available: instead on relying on the '`editable`' parameter (which will always expect a primary key), you can implement a custom edition function, using the '`editfunc`' parameter. Doing so offers the room to implement other ways of identifying a row, e.g. by matching values in multiple columns at the time, so as to emulate a composite primary key (see section **Table-body, Some basic cell configuration** for example code).

Another remark is about the sorting parameter. In the previous example, we've defined sorting in the table-wide setting array. Of course, deciding if some setting belongs to the table-wide settings or to the column settings, is not always self-evident. For example, sorting a table by a given column can be seen as a column setting instead of a table-wide setting. Which is the reason why the API provides us with a parameter to define sorting at column level too:

```
oTableConfigurationList = {

       "my_table": {

              "id": {
                     "colsort": "asc"
              }
       }
};
```

But this parameter – dating from the early Lex'it days – is not recommended: you might want to sort a table by multiple columns at the same time, that is: by a primary column and a secondary column (or even more columns). For example by a person's last name (primary) and then by their first name (secondary). In the past, that was achieved by ordering the columns settings of sorting columns in

the `oTableSettingsList` by priority: primary sorting columns first, followed by the secondary sorting column, etc. Of course, this can be neatly done with a table-wide parameter, which is now the recommended way:

```
oTableSettingsList = {
	"some_other_table": {
		"columns_sorting": {"lastname": "asc", "firstname": "asc"}
	}
};
```

Call your Lex'it project via a URL

You can open Lex'it straight in a given project (that is: skipping the menu page) by specifying the projectname in the URL:

```
http://<your_host>:8080/lexit2/?db=myproject
```

Full list of configuration parameters

oTableSettingsList parameters

Key name	Value type	Description
`callback`	`function`	callback to be executed each time the table is drawn (initialisation) or redraw (when changing page or refreshing) `callback: function(t){ do something }`
`close_callback`	`string`	callback to be executed when the Close button in clicked
`columns_button`	`boolean`	display/hide the "Column selection" button (default: true)
`columns_order`	`array`	Change the default columns order (default: null) `columns_order: ["colname1", "colname2", "colnameX"]`
`columns_sorting`	`array`	Declare the table default sorting. Each column and sort direction is to be given a key/values in an associative array: `columns_sorting: { colname1: sortdir1, colname2: sortdir2, ...}`
`compact_footer`	`boolean`	Make the table footer a bit more compact, eg. by putting the export buttons on the same line as the pagination buttons. (default: false)
`contextmenu`	`function`	Set a context menu at row/cell click The contextmenu-object has to be define following this API: https://swisnl.github.io/jQuery-contextMenu/docs.html
`destroy_callback`	`function`	callback to be executed as soon as a table is destroyed.
`displaylength`	`string`	By default, a table shows 10 rows at the time. Change this number with this setting. `displaylength: 5`
`displaylength_menu`	`string`	Modify the 'Show ... rows' menu (in the table header). For 'show everything' use value -1. `displaylength_menu": [1, 5, 10, 50]`

Key name	Value type	Description
`drag_callback`	`function`	Callback to be executed after a table has been dragged
`draggable`	`boolean`	Make a table draggable (default: true)
`export_buttons`	`boolean`	Display the export button at the bottom of the table (default: true)
`exact_count`	`boolean`	Tell if Lex’it must return an exact table (results) count (default: false)
`focus_at_mouseover`	`boolean`	Give a table focus (set it as active) if the mouse enters its container (default: false) See `fn.setActiveTable()` function
`footer_height`	`string`	Height of the footer under the table (default: 50px)
`formgrid`	`object`	Declare a form as a custom view for a table (see **Form views** section)
`full_export_button`	`object`	Add a full export button to the table footer (see **Add a full export button** section)
`get_focus_on_tab`	`boolean`	Set if a table is supposed to get focus when pressing TAB (default: true)
`goto_button`	`boolean`	display/hide the "Go to" button (default: true)
`group`	`string`	You might wish to give the tables selection menu a bit of structure. That can be done by gathering the tables that belong together, and display those as a group under a particular label. To do so, assign each table a label with the 'group' parameter. All tables sharing the same label value will be shown together under this group name.
`grouping_column`	`string`	When a table contains a column by which rows can be grouped (for example a column *bundle_title*, while rows are about articles as parts of bundles), the column name can be given as a value to the “grouping_column” param. Beware: this parameter allows for showing grouping labels, not for the grouping itself. The table needs to be sorted by the group column to actually ‘group’.

Key name	Value type	Description
header_color	string	By default, the header of a table has a light grey background color. Give it another color by assigning a color code to 'header_color’. With value 'auto', Lex’it chooses a color automatically
header_height	string	Height of the header above the table (default: 50px)
help_button	boolean	display/hide the "?" button (default: true)
info	string	If you want to show more info about a tablet in the tables selection menu, this info can be declared here. When set, the information will be seen in the ‘Choose a table’ menu.
keep_small	Boolean	Keep the height of the table as small as the content allows (default: false)
keydown	function	Assign functions to keys at keydown event
keyup	function	Assign functions to keys at keyup event
left	string	Horizontal position of the left upper corner of the table (px)
main_search	boolean	Set if the global search box (which searches each column of the table) needs to be available (default: true)
menu	array	Set a pulldown menu for a button in the table header
nice_name	string	Give a table a user friendly name (instead of the possible technical sounding name it might have).
pagination_at_bottom	boolean	Display pagination at bottom of the table content (default: true)
pagination_on_top	boolean	Display pagination on top of the table content (default: true)
pagingtype	string	Set paging type of a table Like in: https://datatables.net/reference/feature/paging.type (default: full_numbers)

Key name	Value type	Description
`preinit_callback`	`function`	callback to be executed before a table is shown and its data loaded (this allows to performing some data update or so before the table is shown and populated).
`prereset_callback`	`function`	callback to be executed when the Reset button in clicked (can be used to ensure some filters are reapplied etc.)
`refresh_button`	`boolean`	display/hide the "Refresh table" button (default: true)
`refresh_upon_focus`	`boolean`	Refresh a table when the window regains focus (default: true)
`repeat_callback`	`boolean`	Repeat the callback each time the table is being redrawn (default: false)
`replace_button`	`boolean`	display/hide the "Search & Replace" button (default: true)
`reset_button`	`boolean`	display/hide the "Reset" button (default: true)
`resizable`	`boolean`	Make a table resizable (default: true)
`resize_callback`	`function`	Callback to be executed after a table was resized
`save_callback`	`function`	Callback to be executed after a form was saved
`selected`	`string`	Pre-select an item, in the list of items declared with the ‘menu’ setting.
`selection_button`	`boolean`	display/hide the "Row selection" button (default: true)
`selection_button_active`	`boolean`	Activate the "Row selection" button of a table as soon as it’s opened (default: false)
`size`	`string`	Width of a table. Synonym: `"width"`
`top`	`string`	Vertical position of the left upper corner of the table (px)
`undo_button`	`boolean`	display/hide the "Undo" button (default: true)
`viewtype`	`string`	Set the view mode of a table: 'table' (default) or 'form'.

Key name	Value type	Description
`viewtype_button`	`boolean`	display/hide the "Form/Table view" button (default: true)
`width`	`string`	Width of a table. Synonym: `"size"`

oTableConfigurationList parameters

Key name	Value type	Description

`bgcolor`	`string`	Assign a background color to a column. You can assign both a single or a pair of colors. Assigning only one color implies that Lex'it will have to compute automatically a second, complementary color, because different colors must be shown on even and odd rows, making the screen more readable. When assigning two color codes, those will be used for even resp. odd rows, and Lex’it won’t need to compute a second color.
		`bgcolor: "grey"`
		`bgcolor: "#123ABC"`
		`bgcolor: ["#123ABC", "#456DEF"]`

`button`	`string`	Put a button in a column, and give it the name specified as a parameter value. Instead of setting a name, you can instead have the button use the column’s textual content as a label. To achieve that, give this parameter an empty string as a value. `button: "Click me"`
`button_tooltip`	`string`	Set a tooltip for a button, to be shown at mouseover. `button_tooltip: "some explanations…"`
`cell_tooltip`	`string`	Set a tooltip for a cell, to be shown at mouseover. If none is set, the column name and row number will be shown by default (which can be disabled by setting `cell_tooltip:false`) `cell_tooltip: "click here to edit"`
`choosefrom`	`string`	Set the search box of a column to be a select-box. The values to choose from, can be given in two ways: [1] as an empty array, in which case Lex’it will gather the unique column values in the database to build the select box. E.g.: `choosefrom: []`

Key name	Value type	Description
		[2] as an array of strings. Since the select box has “Please choose” as a first value, it must have an underlying neutral value too (which corresponds to the database search value for this select item). This neutral value must be given as the very first member of mentioned array; an empty string will do. E.g.: `choosefrom: ["", "1", "2", "3"]` in which `""` is the underlying neutral search value corresponding to `"Please choose"`.
`choosefrom_labels`	`object`	Set the labels for the items of a select box. This is needed when the database returns technical sounding values to build the select box with, while you want to have human readable values instead. Like: `choosefrom_labels: {"val1": "label1", "val2": "label2"}}`
`class`	`string`	Assign a CSS class to a column, so as to be able to assign styling properties. That is: classes declared in `css/lexit_table.css`. Lex'it has several default classes available, such as 'inlfont10pt' (to allow rendering of INL font in 10 pt), or 'inlfont' (this one has no font size specified). Note that you can easily add a class by using the `fn.addCss()` function.
`click`	`function`	Assign a function to be executed at cell click `click: function(t, n, e){}`
`colsort`	`string`	If given as a setting for a column, the table will be sorted by this column right at initialization. The allowed values are `'asc'`, `'desc'` or `null` (default). Beware: for multilple columns sorting, use the `"columns_sorting"` `oTableSettingsList` parameter instead.
`contextmenu`	`object`	Set a context menu, to appear when right clicking a cell. The contextmenu-object has to be define following this API:

Key name	Value type	Description
		https://swisnl.github.io/jQuery-contextMenu/docs.html
copy_upon_insert	boolean	If set to true, the Search&Add action (in Search and Replace screen) will use this column’s value for insertion. Columns having this parameter set to false will instead NOT have their value used at insert.
dblclick	function	Assign a function to be executed at cell doubleclick `dblclick: function(t, n, e){}`
editable	boolean	Set a column to be manually editable. **BEWARE**: this parameter only works when the database table has a primary key defined. `editable: true`
editcallback	function	Callback function to be executed right after a cell was manually edited. Beware: this is different from 'editfunc'. 'editfunc' is meant to modify the default behavior of ‘editable’. Instead 'editcallback' is executed as a final step, after 'editfunc'. `editcallback: function(t, n, newValue){ doSomething(newValue); }`
editerrorhandler	function	Handles to be executed as soon as an error occurs in an editable cell. `editerrorhandler: function(err){ console.log(err); }`
editfunc	function	Set a custom function to process cell edition. By default, Lex’it would take the user input and update the corresponding database cell with it. ‘editfunc’ can be defined to have Lex’it this processed otherwise. `editfunc: function(t, n, newValue){ doSomething(newValue); }`

Key name	Value type	Description
editpreprocess	function	Set a function to be applied to values entered when editing a cell. That way, a value can be modified upon edition, before it is inserted/updated in the database. `editpreprocess: function(value){ return value.replace(/thing/, ""); }`
editrefresh	array	Set a list of tables to be refreshed after a cell was edited. (deprecated)
edittrigger	string	As a kind of data validation, you can set a regex as a ‘edittrigger’. When the user’s textual input in a editable cell matches the regex, the 'editfunc' will be triggered; otherwise it won’t. In the following example, we expect the entered value to contain a pipe ("|"). `edittrigger: "\\\|"`
ellipsis	boolean	Set ellipsis on or off in a cell with textual content. The full content will only be shown at mouseover.
ellipsis_delay	integer	Set a delay for ellipsis to be activated. Setting some delay allows users to go with the mouse over ellipsis cell without immediately getting the full textual content shown. The text will only unwrapped when the mouse keeps on the cell beyond the set delay. `ellipsis: true, ellipsis_delay: 500`
ellipsis_height	string	'max-height' attribute for ellipsis. When the cell content goes beyond this value, a scollbar will be added to the ellipsis cell. `ellipsis_height: "200px"`

Key name	Value type	Description
`ellipsis_keep_selected_unwrapped`	`boolean`	When true, an unwrapped ellipsis cell will remain open a mouseout; when false, it won’t. (default: true)
`ellipsis_unwrap`	`boolean`	When true, the textual content of a cell will be unwrapped at mouseover; when false, it won’t.
`ellipsis_width`	`string`	'max-width' attribute for ellipsis. `ellipsis_width: ($(window).width())*.33`
`filter`	`string`	Set a filter to be applied at initialization time. Beware: to keep the filter active afterwards, you’ll need to set `'keepfilter':true`. `filter: 567`
`flexible_visibility`	`boolean`	When 'false', a user won’t be able to change the visibility setting of this column (this can be used when some info has to be kept hidden). `flexible_visibility: true`
`keepfilter`	`boolean`	When 'true', a filter (from parameter 'filter':...) will be reapplied when the Reset button is pressed.
`nice_name`	`string`	Give a column having a technical sounding name, a human readable / user-friendly name. The list of column names can be retrieved by calling the Lex’it internal register function `mt.getListOfColumnsOf( t )`. But if you want to get this list with all names converted into ‘nice names’ (when available), `fn.getListOfColumnsNiceNames(t)` is the function to be called instead.
`query_builder_allowed`	`boolean`	If this is 'false', the query builder will be disabled by CTRL+click in the search field.
`query_builder_values`	`object`	Declare the search values (keys) to be shown in the query builder, and their translation into

Key name	Value type	Description
		database-understandable values (values) to construct a well-formed query.
`query_builder_processor`	`function`	Function to modify the query formulated by the query builder before it is sent to the database.
`query_builder_settings`	`object`	Settings for the query builder (if the default behavior needs to be adjusted). Parameter 'grid': true/false determines whether the search values should be displayed as a grid. Parameter 'preselected' specifies which value should be preselected in the query builder. Parameter 'autostart': true/false determines whether the query formulated by the query builder should be immediately sent to the database when clicking OK.

`render`	`function`	Modify the content of a text cell upon rendering. This function has no influence on the underlying data.
		`render: function(text, nRow){`
		`  return text.toUpperCase()`
		`}`

`searchable`	`boolean`	Set a column to be manually searchable (default: true).
`searchform`	`function`	This parameter is **deprecated**, use `"searchpreprocess"` instead
`searchpreprocess`	`function`	Set a function to be applied to some filter value as soon as a search is started: this can be used to automatically replace some search terms by other, etc. to ensure a successful search. Synonym: `"searchform"`
`select_trigger_event`	`string`	An editable cell where only a limited number of values are allowed is typically edited via a select box. With this parameter, you can declare which mouse event should trigger the

Key name	Value type	Description
		display of the select box. The default is 'mouseover'.
`sortable`	`boolean`	Set a column to be manually sortable (default: true).
`textcolor`	`string`	Set the font color in a column.
`textfont`	`string`	Set the font-family in a column.
`textsize`	`string`	Set the font-size in a column.
`textstyle`	`string`	Set the font-style in a column.
`textweight`	`string`	Set the font-weight in a column.

`validator`	`string`	Set a key that HAS to be pressed when using a select menu in a cell. This can be convenient to prevent selection of an item by mistake when moving the mouse or so. Pressing the key indicates that the user is selecting on purpose.
		`"some_column": {`
		`editable: true,`
		`validator: "shift"`
		`}`

`visible`	`boolean`	Set the visibility of a column. To prevent the user from modifying the default setting, set `"flexible_visibility": false` as well!
`width`	`string`	Set the width of a column. This can be specified in 'px', or as a relative value (in '%' of the table width).

Giving access to others

Assign rights to users

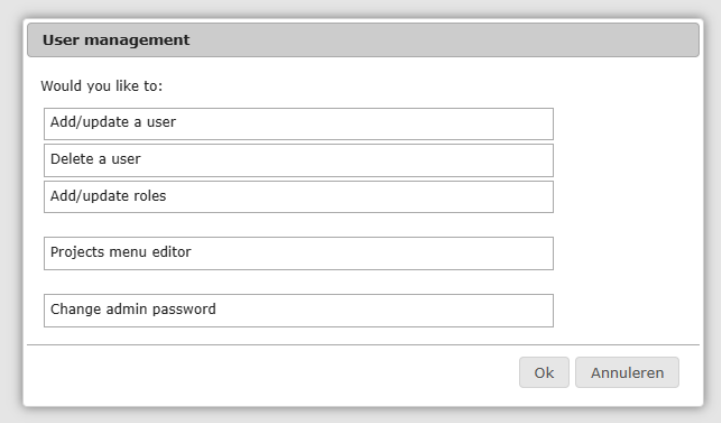
In the 'Installation: quick start' section, we already created your own user. But as an admin, you'll probably need to create other user accounts and assign them particular roles.

Setting rights

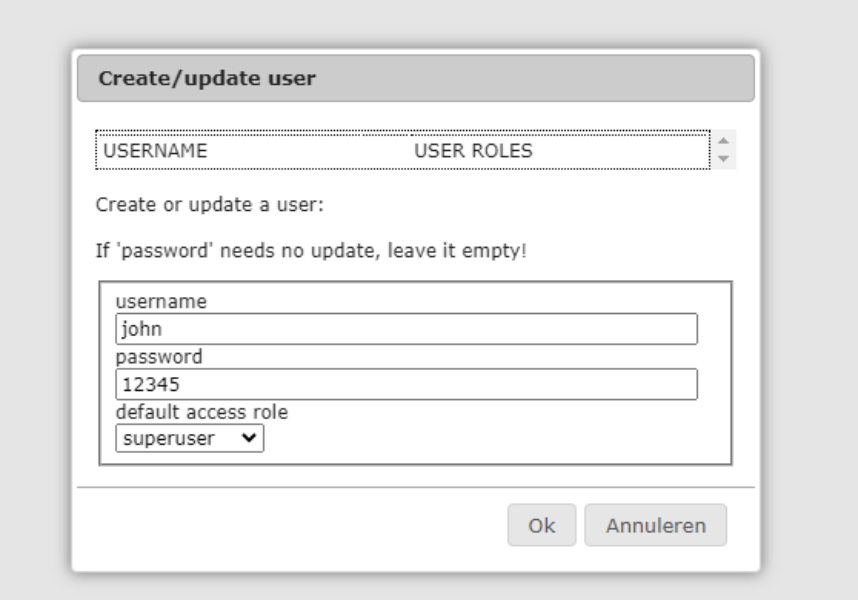
Go to the 'admin' screen by typing this URL:

```
http://<your_host>:8080/lexit2/?db=admin
```

Log in with the admin password, and the following screen will open:



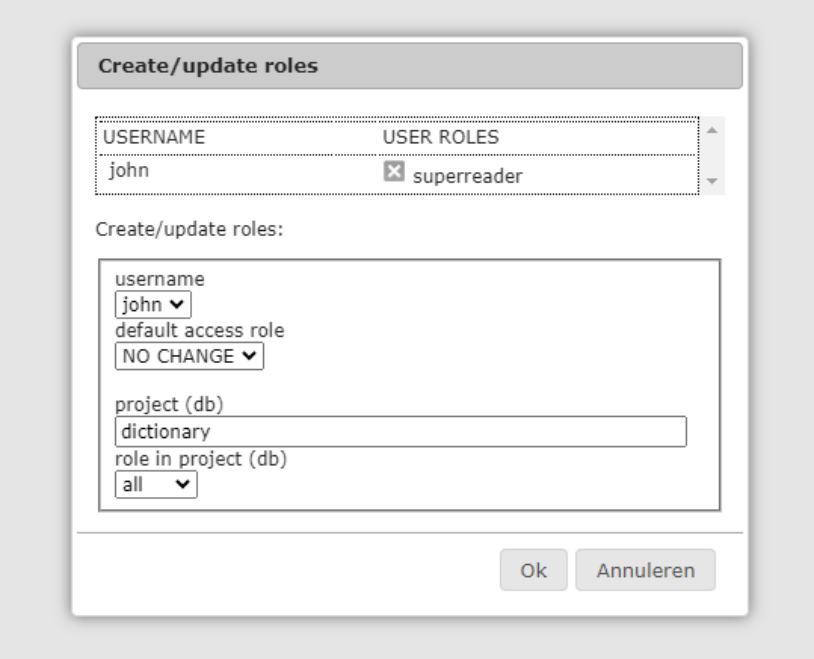
Now, let's say you want to add the user 'john'. Click onto 'Add/update a user'. Now type in the username 'john' and some password:



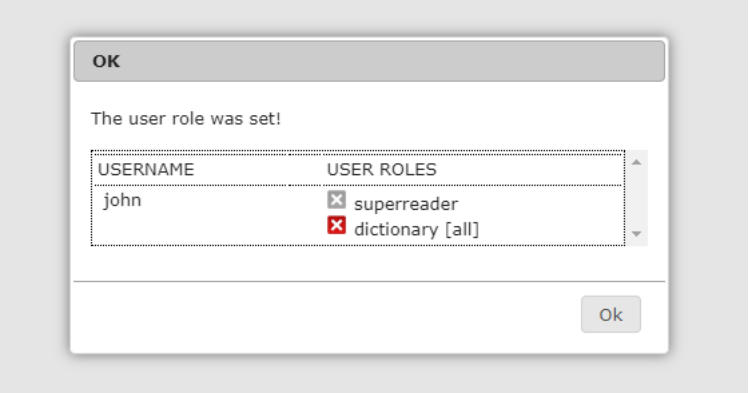
You might want to set a default access role, though that is not required at all. Defining a default access role is just a quick and easy way to grant access, without specifying any limitation to a particular project. Two different default access roles can be given:

(1) The 'superuser' default access role gives full access rights, like reading, writing and deleting data in any project.
(2) The 'superreader' role gives the right to access projects for reading only, so the user won't be able to edit nor delete anything.

Next to default access roles, users might be assigned specific roles in specific projects. For example, we could give our user John 'superreader' rights to allow him to view any project, but a particular project might require John to have more rights like writing and deleting.
That's where the 'Add/update roles' button comes in. Clicking that button will open the following screen:



To be able to set John's rights, choose his name in the 'username' select box.
Now, let's say the project John needs writing/deleting rights for is called 'dictionary'. Type in the project name in the 'project (db)' box.
Finally, set the role to be granted in that project: 'all' (read/write/delete), 'write' (read/write) or 'read' (read only!). Let's grant John full rights for the 'dictionary' project. As soon as you clicked 'OK', you should get this confirmation dialog:



John now has reading access in any project by default, but has all rights in the 'dictionary' project.

Show your projects in a menu

When accessing Lex'it through its main URL (`http://<your_host>:8080/lexit2/`), the application shows a main menu. This menu displays the available projects: for each project a short description, and a clickable project name. Clicking a project name gives access to that project.

Of course, the main menu requires a bit of configuration.
Two possibilities are offered:

[1] the menu can be configured in a file named `projects_overview.js`
(see the **My first project** section for its location)

[2] the menu can be configured in the Admin GUI
(see 'Access the admin GUI' in the **Installation: quick start** section). This is now the recommended method.

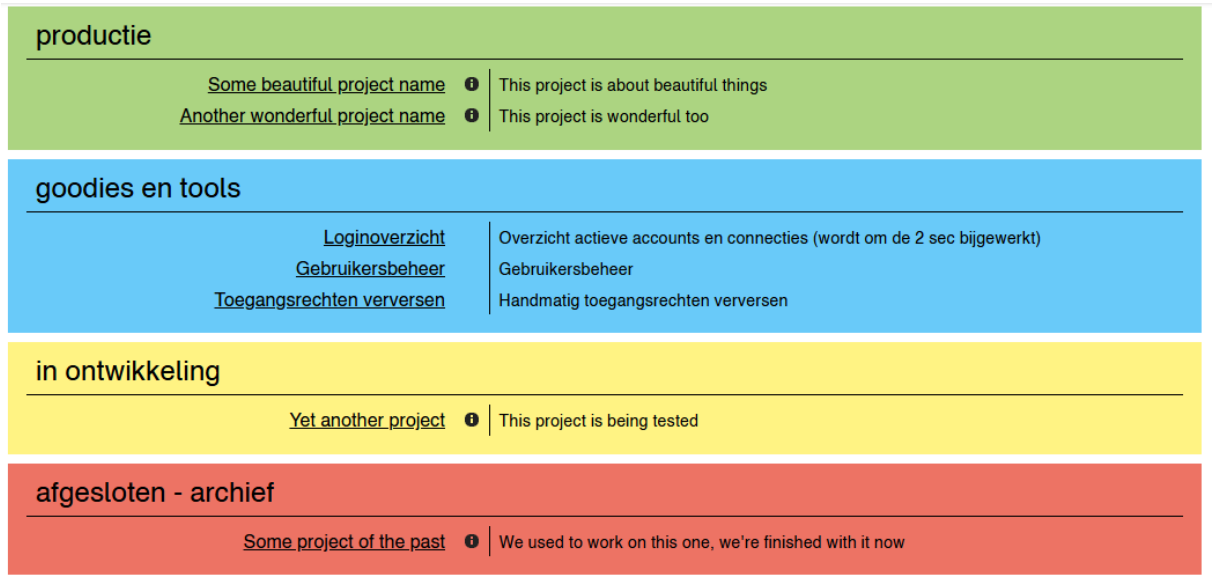
Use the projects_overview.js file

Although this is not the recommended method anymore, this way of configuring the projects menu is still default. That is: when the projects menu is being accessed, Lex'it first tries to retrieve the `projects_overview.js` file so as to get the needed information from it. But when this file is missing, Lex'it falls back to the second (and recommended) method: it calls the webservice to retrieve the information stored in the Admin GUI.

Here are a few examples of projects metadata declared in `projects_overview.js` for display in the projects menu. Please note that the `projects_overview.js` file contains a header with more documentation about how to declare your projects metadata.

```
var aProjectList = [
	{
		name: "Some beautiful project name",
		config_filename: "my_configfile",
		description: "This project is begin used in production",
		production: true
	},
	{
		name: "Yet another project",
		config_filename: "my_dev_proj_configfile",
		description: "This project is being tested",
		production: false  // false is actually default!
	},
	{
		name: "Some project of the past",
		config_filename: "my_old_proj_configfile",
		description: "We used to work on this, but we're finished now",
		closed: true
	},
	…
```

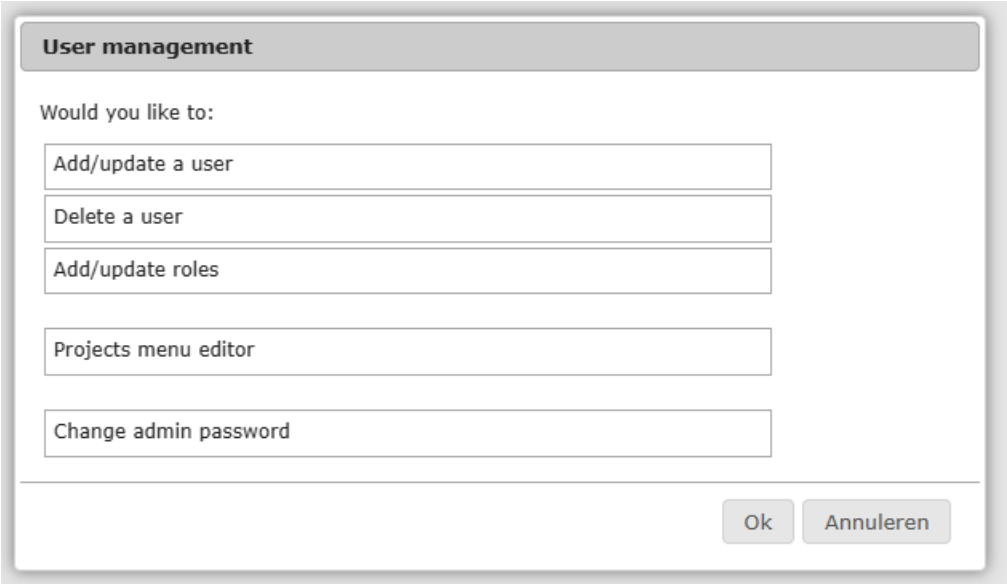
These lines will result in the following projects menu:



Use the Admin GUI

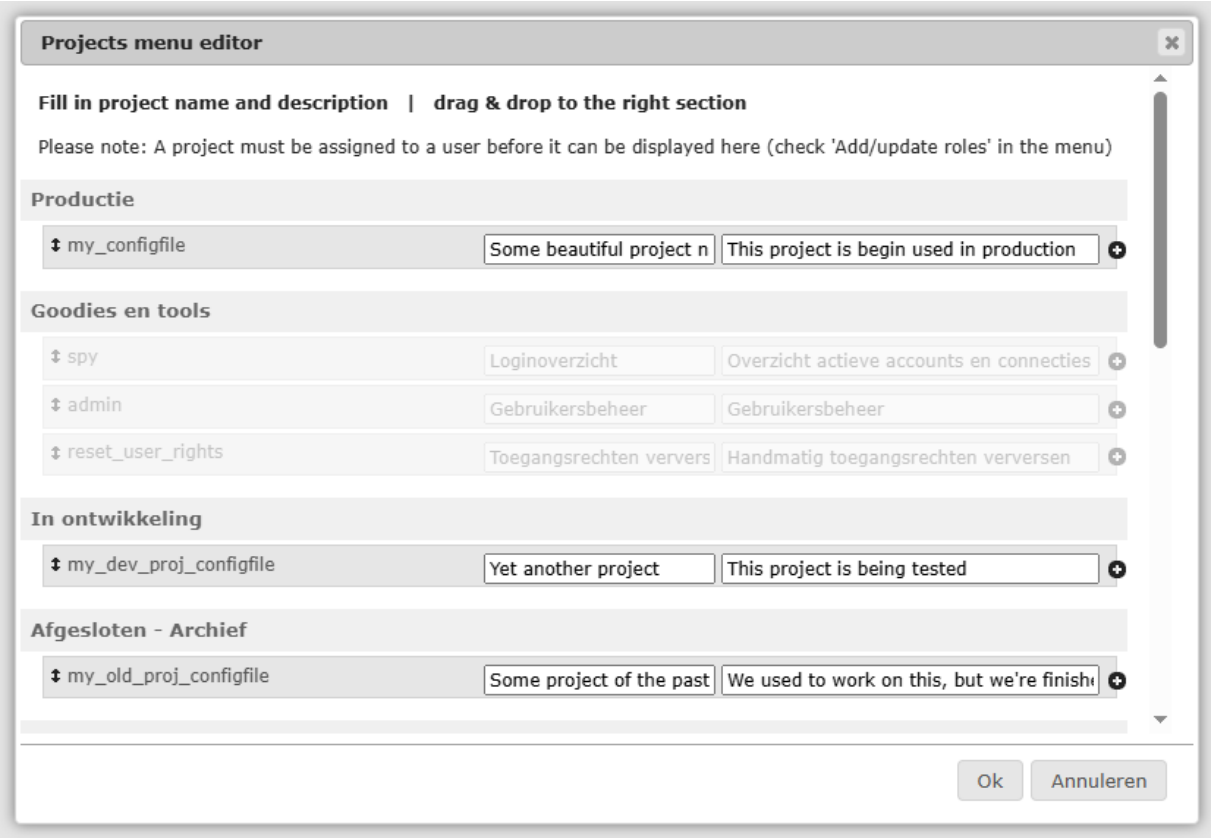
As said hereabove, the recommended way to configure the projects menu is by using the Admin GUI (don't forget to remove any `projects_overview.js` file, otherwise Lex'it will use the information from this file instead of retrieving information from the Admin GUI).

Once you've access the Admin GUI (as described in 'Access the admin GUI' in the **Installation: quick start** section), we'll get the following menu:

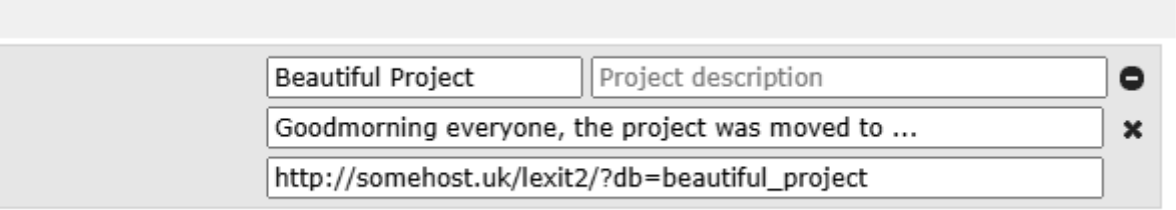


Click on the 'Projects menu editor' option to access the editor. That is the place where your projects can be configured.

In the previous section 'Use the projects_overview.js file', we showed some sample metadata as an example of a projects menu configuration. If you happen to use the same sample metadata in the projects menu editor of the Admin GUI, we will end up with the following screen:



Please note that any project can be dragged & dropped from one section to the other (eg. from development to production, etc.). Additionally, clicking the ‘+’ sign shows extra metadata fields, in which a ‘Message to users’ can be typed in (which, as you might suspect, will be shown to users when they access the project), and also a ‘Redirect URL’ (which is sometimes needed, when you want to allow your users to access a project using the link they’ve always used, but want to force them to use another Lex’it instance instead):



At last, a project can be removed from the projects menu by clicking the ‘x’ sign.

The API

The API provides for functions to deal with different aspects of the tables. The most important are:

- The header
- The search boxes
- The table body (cells, rows, columns)
- Dialogs
- Functions and callbacks

The Lex'it API is built upon the DataTables API, which means that not only the Lex'it functions but also all the native DataTables functions can be used. Beware that Lex'it might not be using the very last version of DataTables. Check the following file: its headers tells which version Lex'it is running now.

```
…\webapp\js\jquery.dataTables.js
```

The current DataTables API is decribed at: https://datatables.net/reference/api/

First things first: open a table

A table can be opened the following way:

```
fn.callTable(tablename);
```

Of course, you can specify some filtering, and even give a callback to be executed as soon as the table is opened and initialized:

```
fn.callTable(tablename, {"somecolumn_name": value}, function(){
       // do something when table is open
} [, oExtraSettings]);
```

The `oExtraSettings` parameter is optional, but allows to set the position of a table, its view mode, its display length etc. from the very start. Here are the available parameters, all to be declared in an associative array:

Parameter	Type	Default value	Description
`pane`	`string`	`all elements`	Elements to use in the rendering of the top pane. This string consists of a combinations of letters symbolizing the needed functionalities: 'i' for table information summary, 'f' for filtering input, 'l' for length changing input control, 'p' for pagination control.
`top`	`string`		Vertical position of the left upper corner of the table (px)
`left`	`string`		Horizontal position of the left upper corner of the table (px)
`viewtype`	`string`	`table`	'table' or 'form'
`displaylength`	`int`	`10`	Number of rows to show at once in the table

ignore_ initialisaton _filters	boolean	false	if true, the default filters (stated in config file) to be set at initialization time will be omitted
export_buttons	boolean	true	If false, the export buttons won't be shown.
pagination_on_top	boolean	true	If false, the pagination buttons on top of the table won't be shown.
pagination_at_bottom	boolean	true	If false, the pagination buttons at the bottom of the table won't be shown

This function will open a table in the current tab. If you want to open a table in a new tab, use the following function:

```
fn.callTableInNewTab(tablename, {"somecolumn_name": value}, oExtraSettings,
projectName);
```

The filters applied at the first call of a table can be retrieved by the following function. This is a way of retrieving the initial state of a table:

```
var oFilterValues = fn.getFiltersUponCall(tablename);

var sValueOfSomeColumn = oFilterValues["somecolumn_name"];
```

Close a table

Of course, if we can open a table, we can close it too!

```
fn.closeTable(t, fnCallback);
```

'Mass closing' is possible too. This function will close all tables and call a callback afterwards:

```
fn.closeAllTables(fnCallback);
```

Or you can close all tables but the one specified, and also call a callback afterwards:

```
fn.closeOtherTables(t, fnCallback);
```

Project-wide settings

Setting a project title, background-color, font, etc.

Usually, the title shown in the Lex'it GUI is the content of the `"name"` parameter assigned to the project in file `/webapps/lexit2_config/projects_overview.js`. But if needed, it is possible to modify the title shown programmatically:

```
fn.setProjectTitle(sProjectName, sColor, sFontSize, sFontWeight);
```

A background color can be set for the screen:

```
fn.setBackgroundColor(sColor);
```

It is also possible to set a particular font, to be used project-wide (in tables, tooltips, etc.)

```
setProjectFont(sFontFamily, sFontSize);
```

Or you can apply the IvdNT style ('balk' on top):

```
fn.setBalk(true);

// by default, the buttons in the balk have no function attached.
// if you want to assign the native Lex'it help text to the 'Help' button of the
'balk', call the function with an extra argument:

fn.setBalk(true, true);

// if you want to use a custom project icon, specify its location and size
// (since an icon is square, the given size will be applied to all four sides)


fn.setBalk(true, true, "/path/to/icon.jpg", "52px");
```

Setting or getting the project language can be done with the following functions:

```
lang.setLanguage("en");
// now the following should yield 'en'
var sLangCode = lang.getLanguageCode();
```

Available languages are Dutch (value '`nl`', which the default setting) and English (value '`en`').

Other languages can easily be implemented by editing the `lexit.language.js` file:

```
if (sLanguageCode == "nl"){
	// do nothing, since it's default
}
else if (sLanguageCode == "en"){

	// general
	lang.ok = "Ok";
	lang.send = "Send";
	lang.save = "Save";
	lang.undo = "Undo";
	lang.yes = "Yes";
	lang.no = "No";
	…
	…
}
```

Check for page visibility (meaning: is the browser tab active / in sight?)

```
var bPageVisible = fn.pageIsVisible();
```

Setting table group in table menu

When a project has lots of tables, it might be convenient to organize the table menu, by grouping the tables that belong together under one single label. To do so, give the table a group name in their setting:

```
oTableSettingsList = {

	"lemmata": {

		"group": "lexicon information"
	}
}
```

The tables menu can be reorganized at runtime. For example, to assign the 'lemmata' table hereabove to another group called 'some_other_group', just run the following code:

```
fn.closeTable("lemmata", function(){

	fn.assignTableToGroup("lemmata", "some_new_group");
	fn.rebuildTablesMenu();
});
```

If you wish to put a table back to the default group, replace `"some_new_group"` by `null`.

Table global settings

General settings

Get the list of tables to choose from:

```
Var aListOfTables = fn.getListOfTables();
```

Is a table loaded?

Do you need to know programmatically if a table was loaded already? Do this:

```
if (fn.tableExists(t) ) { … }      //  synonym: fn.tableIsOpen(t)
```

Get the name of a table

You can retrieve the name of a table, given any piece of information about it (a cell or row node, or a API object instance):

```
var sTableName = fn.getTableName(mixed);
```

You can also modify the name of a table programmatically, that is: modify the name that is rendered in the table header. Note that using the `"nice_name"` parameter in the table settings is the only truly persistent way to do this:

```
fn.setTableNameInHeader(sTableName, sNameToShowInTheGUI);
```

View type

The Lex'it GUI gives two viewing type for table content: 'table' view or 'form' view. This is set manually by the user. Which mode is on, can be read by the following function:

```
var sViewMode = fn.getViewType(t);
if (sViewMode == 'form') {

       fn.message("OK", "You're viewing the table in form mode now!");
}
```

You can easily switch to the other view type:

```
fn.toggleViewType(sSomeTablename, fnCallback);
```

Row selection

The Lex'it GUI has a 'row selection mode', that can be switched on or off by the user. When turned on, a user can click rows for selection without triggering function usually attached to rows (like editable, click, etc.). Check if this mode is on or off by calling:

```
fn.rowsSelectionIsAllowed(t);
```

Is a table empty?

A reliable way of knowing programmatically if a query has returned any results, is by calling the following function. You would typically call it inside the draw callback of a table (`callback` parameter in the table settings), which is fired after a query was performed:

```
if (fn.tableIsEmpty(t) ){ … }
```

Table results count quality

Lex'it always tries to count the number of results as fast as possible. To do so, it will often choose to compute an estimated count instead of an exact count, which is much, much faster. However, this might sometimes not be convenient. If you need Lex'it to (always) return exact counts instead of (much faster) estimated counts, proceed this way:

```
oTableSettingsList = {

       "some_table": {

              "exact_count": true
       }
};
```

Positioning

Table absolute positioning

You can set the position of a table by dragging it with the mouse, or you can set the absolute screen position programmatically.

Setting the position programmatically can be done as part of the table call:

```
var oExtraSettings = {top: …, left: …};
fn.callTable(t, {"somecolumn": value}, fnCallback, oExtraSettings);
```

…or any time after calling the table:

```
fn.putTableAtPosition(t, aPos);

// aPos can be an associative array {left: x, top: y},
//          or just an array of integers [x, y]
```

And of course you can read out the table position:

```
var aPos = fn.getTablePosition(t);

// this returns an associative array like: {top: …, left: …}
```

It is also possible to center a table horizontally on the screen:

```
fn.centerTable(t);
```

Table relative positioning

You can set the position of a table relatively to other tables. For example, you can align two tables horizontally, in such a way that those appear side to side:

```
fn.alignTables(t1, t2, fnCallback);
```

There also exists a vertical equivalent, allowing to put two tables on top of each other:

```
fn.pileupTables(t1, t2, fnCallback);
```

Another aspect of relative positioning is z-index. To put a table in front, just call:

```
fn.putTableInFront(t);
```

If needed, you can also check programmatically which table is in front at the moment:

```
var sTableName = fn.getTableInFront();
```

Saving and retrieving settings

Read the configuration settings

In the **My first project** section of this document, we saw that configuration is set in two main arrays, one for table wide configuration and one for column level configuration.

The *table wide* configuration can be retrieved by:

```
var oTableSettings = conf.getTableSettings(sTablename);
```

The *column level* configuration is to be retrieved by:

```
var oTableConfig = conf.getTableConfig(sTablename);

var oColumnConfig = conf.getColumnConfig(oTableConfig, sColumnName);
```

Restore the configuration settings

Programmatically click on the reset button in the table header:

```
fn.resetTable(sTableName, fnCallback);
```

Save table extra settings (position etc.) for later re-use

Tables settings consist of its position (`{top: … left: …}`), its view type (table, form), its display length, its size.
When a table needs to be closed, and you want to be able to re-open it with the very same settings (like screen position, etc.), this is what needs to be done:

```
// save the table settings before closing your table:
var oExtraSettings = fn.getTableExtraSettings(sTablename)

// now close it
fn.closeTable(sTablename);

// do other cool stuff
// …

// open the table again with the settings saved before:

fn.callTable(tablename, {"somecolumn_name": value}, function(){
       // do something when table is open
}, oExtraSettings);
```

Save and restore the table state

With table state, we mean: the active sorting columns, the active filters (=search values) applied to the table, and its display position (shown page). It is possible to save this table state and restore it later on:

```
// save current table state
fn.saveTableState(sTableName);

// do something else with the table
// …

// restore the original state of the table
fn.restoreTableState(sTable);
```

Respawn a table

If a table has been closed, and you want to re-open it and have it back in its last known state, you can use this function:

```
fn.respawnTable(sTableName);
```

By default, Lex'it will restore several settings: `"top", "left", "displayRow", "displaylength", "order", viewtype", "searchFilters"` and `"width"`. If you need to know which settings are available for restoration, you can check the output of this function, which retrieves the full table state (when the table is open):

```
var oState = fn.getTableState(t);
```

If you want to respawn a table but prevent some settings from being restored, just give the names of the settings to exclude from restoration as a second parameter:

```
fn.respawnTable(sTableName, ["viewtype", "width"]);
// viewtype and width won't be restored
```

And of course, you can set a callback too:

```
fn.respawnTable(sTableName, null, function(){
 …
});
```

Retrieve the table wrapper - Access table HTML elements

The table wrapper is the `div` containing the header of a table (`top` class), its body (`dataTables_scroll` class) and its export pane (`export_pane` class). The node of the table wrapper can be retrieved by calling the `fn.getTableNode()` function. This gives access to the table HTML elements, and allows you to apply CSS or some JavaScript functions:

```
var nWrapper = fn.getTableNode(x); // table name, any cell or row node, etc.

// do anything (eg. apply css to the table header)

$(nWrapper).find( ".top" ).css(…);
```

Table header

Ways to set header buttons

In Lex'it, header buttons come in two flavors:

- Buttons to be appended to the default Lex'it buttons, which we call 'custom buttons'
- Buttons to be appended to any position in the header, which we call 'free buttons'

Custom buttons

Usually, buttons are declared in the `oTableSettingsList`. Such buttons will be available in the GUI from initialization time:

```
oTableSettingsList:{

	"button_0": {
		"name": "put the button label here",
		"click": function(t){
			// do something
		},
		"tooltip": "Click this button to do cool things!"
	}
}
```

However, it is also possible to add buttons later on. In that case, the buttons name and behavior have to be set into an associative array, which can be given as an argument to the following function:

```
var oButtonConfig = {

		"name": "put the button label here",
		"click": function(t){ … }
	};

fn.addCustomButton(sSomeTableName, oButtonConfig);
```

Styling a button

A button can be assign a text color, a background color, or even a class:

```
// declare a class to be used in the button declaration
fn.addCss(".classname {background-color: #000000, …}");

oTableSettingsList:{

	"button_0": {
		"name": "put the button label here",
		"click": function(t){},

		"class": "classname",  // use the declared class

		// or you could do this instead:
		"textcolor": "#FFFFFF", // black button with white font color
		"bgcolor": "#000000"
	}
}
```

Add a SELECT button

Give the button a `name` and define the options of the select menu by giving up each option name as a key, and its behavior as a function (with the table variable as a parameter):

```
"button_0": {
       "name": "Name of the button",
       "menu": {
              "option name 1": function(t){
                     // do something cool here
              },
              "option name 2": function(t){
                     // do something else here
              },

              // if needed, set option to be selected by default
              "selected": "option name 2"
       }
}
```

Implement a button for row duplication

Row duplication can be implemented in a generic way by using the Lex'it native function `fn.duplicateRecord(sSomeTable, aListOfColumnsToSkip, primaryKeySubstitute, primaryKeyValue, fnCallback)`.

Duplicating a row doesn't necessarily imply duplication of each column of that row: some columns need to be skipped (like the primary key, obviously). Columns excluded from duplication can be declared in an array as second parameter of the function (`aListOfColumnsToSkip`).

Of course, you need to tell the function which row has to be duplicated. That's where the parameters `primaryKeySubstitute` and `primaryKeyValue` come into action. If the table has a primary key set in PostgreSql already, `primaryKeySubstitute` can be left empty (`null`) and you only need to specify the row ID in the `primaryKeyValue` parameter. However, if the table hasn't any primary key, you must specify a column that can act as such: its name can be given in the `primaryKeySubstitute` parameter:

```
"button_0": {
       "name": "Duplicate row",
       "click": function(t){
        var nRow = fn.getFirstSelectedRowNodeFrom(t);
        var iCounter = fn.getDataFromCellInRowNode(nRow, "counter");

        fn.duplicateRecord(t, null, null, iCounter, function(response){

        // arg #2 with null value means we won't skip any field (that is, the
       whole row with all its cells will be duplicated, except the primary key of
       course)

       // arg #3 with null value means we rely on the primary key ('counter'), so
       we don't need to declared some other serial typed column to act as such
       (which is needed when a table lacks a primary key)

              fn.refreshTable(t, function(){
                     fn.message("New row", "The duplicated row has
                     ID counter="+response);
               });
        });
       }
 },
```

Click on a button programmatically

This can be done by calling this function, specifying the table and the button ID in the DOM tree:

```
fn.clickOnButton(sTableName, sButtonId)
```

Modify the name/style of a button programmatically

The name of a button is normally set in the table declaration in the configuration file. But it might be convenient to be able to change the name of a button. For example, if a button is used to turn on a function, it might be convenient if the button label 'says' that the function is on, etc. That can be done by calling this function:

```
fn.setCustomButtonName(sTableName, iButtonNumber, sNewName);
```

The same can be achieved for the button style:

```
fn.setCustomButtonCss(sTableName, iButtonNumber, sProperty, sNewValue);
```

If you want to read the current style of a button, you should call:

```
fn.getCustomButtonCss (sTableName, iButtonNumber, sProperty);
```

Finally, the index of a button with a given name (or the other way round) can be retrieved with:

```
fn.getIndexOfButtonNamed(sTableName, sName);
fn.getNameOfButtonAtIndex(sTableName, iIndex);
```

Free buttons

The second type of header buttons are the free buttons, called that ways because those can be freely positioned.

Those buttons have to be declared in an associative array "`buttons`", associating button labels to a set of properties:

```
oTableSettingsList:{

	"buttons": {
		"some_button_label": { . . . },
		"another_button_label": { . . . },
		. . .
	}
}
```

By default, the button label acts in the GUI as a button name. But if you wish to have your buttons show another label in the GUI, just add a "nice_name" property:

```
oTableSettingsList:{

	"buttons": {
		"some_button_label": {

			"nice_name": "This will be shown in the GUI"

		},
		"another_button_label": { . . . },
		. . .
	}
}
```

A free button can be given the same properties and the custom header buttons (see that section):

```
oTableSettingsList:{

	"buttons": {
		"some_button_label": {

			"nice_name": "This will be shown in the GUI",
			"click": function(t){ . . . },
			"bgcolor": "yellow",
			"textcolor": "black",
			"class": "some css class",
			"tooltip": "Click here to achieve nice things"
	}
}
```

Additionally, you can set a position, which is an absolute position in the header, with the top left corner of the header being position [0, 0]:

```
      "buttons": {
            "some_button_label": {

                  // this way:
                  "position": ["200px", "50px"], . . .

                  // or this way:
                  "position": {"top": "50px", "left": "200px"}, . . .
      }
}
```

Those buttons can be accessed and modified in ways similar to the custom header buttons. You can set a new button name at runtime, and get/modify its CSS:

```
// setters

fn.setFreeButtonName(sTableName, sButtonLabel, sNewName);
fn.setFreeButtonCss(sTableName, sButtonLabel, sProperty, sValue);

// getter

var sValue = fn.getFreeButtonCss(sTableName, sButtonLabel, sProperty);
var sButtonName = fn.getFreeButtonName(nButtonNode);
```

Get the node of a button, given the table it's in, and the button label:

```
fn.getFreeButtonElementByName(sTableName, sButtonLabel);
```

Get the ID of a button:

```
// given the table name (or its API object instance) and the button label
var sButtonId = fn.getFreeButtonId(mixed, sButtonLabel);

// ...or given the button node
var sButtonId = fn.getFreeButtonId(nButtonNode);
```

Filters & search boxes in the GUI

Data types, operators, etc.

Lex'it supports different kinds of search queries. You can enter search values in the search boxes of the GUI, or you can fire a search programmatically by using functions like `fn.callTable()` etc.

You can search columns with (database) types like: `date`, `time`, `bit varying`, `Boolean`, arrays of textual or numeric type, `json` (JSONB), `text`, `integer`, etc.

By default, Lex'it applies a regular expression operator (`~*`) for text types, or exact equality (`=`/`IS`) for numbers and Boolean. Other operators can be applied by putting them in front of the search value:

Query		Matches…
`!autobus`		Any word not containing 'autobus'
`//autobus`		Exactly 'autobus' (and nothing else)
`^autobus$`		
`>=autobus`	`<=autobus`	'autobus' and any other word following/preceding it alphabetically
`>autobus`	`<autobus`	any word following/preceding 'autobus' alphabetically
`>=10`	`<=5`	10 and all higher numbers / 5 and all lower numbers
`>10`	`<5`	Any number higher than 10 / Any number lower than 5

Since textual types are being applied a regular expression operator by default, you can – of course – use regular expressions:

Query	Matches…
`^aa[^n]`	Words starting with 'aa', except 'aan'
`heid$`	Words ending with 'heid'
`^ge(?!.*heid$)`	Words starting with 'ge-' but NOT finishing on '-heid'
`^(?!ge).*heid$`	Words NOT starting with 'ge-' but finishing on '-heid'

It is also possible to search a column using an array of possible values. Searching an array typed column happens the same way.

Query	Matches…
`{aap, noot, mies}`	Any of the words of the array
`{456,789,2}`	Any of the numbers of the array
`!{aap, noot, mies}`	Any word other than the words of the array

Searching a JSONB column looks the same:

Query	Matches…
`{"editor": "piet", "gender": "M"}`	Any JSON object containing the mentioned properties
`[{"editor": "piet", "gender": "M"}]`	Any JSON object being a member of an array matching the mentioned properties

NOTE when using `fn.callTable()`, the search values must be put between quotes.

Logical operators

When firing a search using `fn.callTable()` etc., a sets of search values will be interpreted as if those values were combined by **AND**-operators.

```
fn.callTable(sSomeTable, { "column1": "value1", "column2": "value2"};
```

So the function call hereabove will be interpreted as a SQL-query like:

```
SELECT *
FROM sometable
WHERE column1 = value1 AND column2 = value2;
```

Now, let's say you need to combine your search filters with an **OR**-operator instead. There are easy ways to achieve that. Which way to choose, depends on what you need to match:

If you need a **column** to match **value #1 OR value #2**, just combine the two (or more) values in an array. Any row in which the column matches one of the values of the array will be returned as a result:

```
fn.callTable(sSomeTable, {"column": "{value1, value2}" };
```

However, if you need **column #1 OR column #2** to match some values, sadly no Lex'it function supports that. So you'll have to write a SQL function to run your query and return the id's of the matching rows, and then have Lex'it call your function and use the returned id's in a `fn.callTable()` call. Here is a quick & dirty example:

```
CREATE FUNCTION api.run_query_with_or_operator(value1 text, value2 text)
    RETURNS text
    LANGUAGE 'sql'
AS $BODY$

       SELECT string_agg(id::text, ',' ORDER BY id) AS ids
       FROM some_table
       WHERE column1 = $1 OR column2 = $2;

$BODY$;
```

Lex'it part:

```
fn.callFunction("api.run_query_with_or_operator", [ valueToMatch ],
function(resp){

       const sArrayOfIds = `{${resp["run_query_with_or_operator"]}}`;
       fn.callTable(sSomeTable, {"id": sArrayOfIds });
});
```

Searchboxes

You can programmatically change the content of a search box, or read what's inside.

First, the search boxes can be reset by:

```
fn.resetAllFilters(sSomeTable, true);
```

And search values can but put into the search boxes by doing the following:

```
fn.putDataIntoFilterBox(sSomeTable, sSomeColumn, sSomeValue);
```

But that will only set the content of the search boxes in the GUI, allowing you to edit this content before starting a search. That means that before a search is started, the engine is NOT set with these search values yet. If you wish to set the engine as well, do this instead:

```
fn.setFilters(sSomeTable, {"column1": value1, "column2": value2}, true);

// The last parameter 'true' tells the function to put the search values in the
search boxes in the GUI as well.
// If set to 'false' instead, the engine will be set but the search boxes in the
GUI won't change.

fn.refreshTable(sSomeTable);
```

The filter behind the 'Search whole table' field can be set too:

```
fn.setGlobalFilter(sTable, sSearchTerm);
fn.refreshTable(sTable);
```

The above example operates **both** on the search engine as on the search boxes (updating the search boxes is actually the meaning of the '`true`' value; set the filters silently with '`false`' instead).

The filters currently active in the search engine can be retrieved by:

```
var aSearchFilters = fn.getFilters(sSomeTable);
```

This function, however, has no access to the search boxes of the GUI.
But their content can be programmatically read by doing the following:

```
var searchBoxValue = fn.getValueOfFilterBox(sSomeTable, sColumnName);
```

And if needed, its type can be retrieved too. The following function will return 'checkbox', 'select' or 'text':

```
var searchBoxType = fn.getTypeOfFilterBox(sSomeTablename, sColumnName);

if (searchBoxType == 'select') {
	fn.message("Info", "You just click on a select box");
}
```

Modify a user's search query programmatically, before search is fired

Sometimes search queries require some pre-processing before the search effectively starts. For example because of the necessity to meet the data format in the database.

Let's say we have a table containing a list of lemmata containing some diacritics (é, à, ü, etc.) coded as HTML entities: searching for a lemma containing a -é- will fail, because the -é- of the search query won't match the equivalent html-entity (that is, `é`). To solve that, you can set the `"searchpreprocess"` function in the table configuration:

```
"lemma": {
	"editable": true,
	"searchpreprocess": function(value){
		value = value.replace("é", "&eacute");
		value = …
		return value;
	}
}
```

In the example here above, the `"searchpreprocess"` parameter makes sure that diacritics occurring in the search term of column 'lemma' are automatically re-written into corresponding html entities.

A noteworthy way of using this function is the following: imagine – for example – you wish to perform an *accent-insensitive* search, that is: searching for 'café' matches both 'café' and 'cafe':[2]

```
"lemma": {
	"editable": true,
	"searchpreprocess": function(value){

		return "unaccent(" + value + ")";
	}
}
```

Another way of modifying a search query is passing a search value to another column. A reason to do so might be that this other column contains an optimized copy of the data to be searched, allowing for a custom way of searching or so.

[2] Of course, this will only work if the 'unaccent' extension has been installed in the database (just type '`CREATE EXTENSION unaccent;`' in a PSQL prompt to do so)

Passing the search value to another column is easy:

```
"lemma": {
       "editable": true,
       "searchpreprocess": function(value){

              // pass search value to other column
              fn.addFilters("tablename", {"other_column": value});

              // cancel search on this column
              return "";
       }
}
```

Modifying the filters silently (that is: without modifying the search boxes)

Filters can be all erased silently (despite the search boxes being filled in!):

```
fn.clearAllFilters(t);
fn.refreshTable(t);
```

To add filters to the table, do:

```
fn.addFilters(t, {"column_name1": val1, "column_name2": val2});
fn.refreshTable(t);
```

The command hereabove won't change the existing filters on other columns. If you need to erase all pre-existing filters, do instead:

```
fn.setFilters(t, {"column_name1": val1, "column_name2": val2});
fn.refreshTable(t);
```

Change the search values of a dropdown searchbox

Let's say you have declared a list of values to choose from in the configuration of a column:

```
tablename: {
	cellname: {
		"choosefrom": ["one", "two", "three"]
	}
}
```

Now imagine you need to add a value to this list some time after initialization (for example because some value was added in the table column, and you want the users to be able to look up that value too!). This is what you'll need to do:

```
// define a new list of values
var aNewListOfValues = ["one", "two", "three", "four"];

// assign this new list of values to the preexisting setting 'choosefrom'
conf.changeTableConfigValue("tablename", "cellname", "choosefrom",
aNewListOfValues);

// redraw the dropdown at once!
fn.rebuildSelectBox("tablename", "cellname");
```

The code hereabove will redraw the searchbox dropdown. But if the column associated to this searchbox was set to be editable, you might want the new values to appear in the edition dropdown too. To achieve that, just call `fn.refreshTable()` as a final command.

Query builder

Lex'it offers query building help. To activate it, strike the CTRL key when clicking into a search box, and the query builder popup will open.

Out-of-the box, the query build gathers the most frequent values of a column and allows to build a query with a selection of those values, combined in a single regular expression. Additionally, you can require exact matching, or negative matching (that is, excluding what matches from the results).

This process can be fine-tuned with some configuration:

The `query_builder_settings` parameter can be used to declare some settings as an associative array:

Parameter name	Default value	Effect
`grid`	`Boolean: false`	Way of rendering the values to choose from: If `false`, show the values as a list. If `true`, show those as a grid.
`preselected`	`Empty array`	List of pre-selected values, to be marked as chosen right from the start
`autostart`	`Boolean: true`	Behaviour after some query was built: If `true`, start the search as soon as OK is clicked. If `false`: the user will have to start the search manually (by pressing ENTER or so)

With the `query_builder_values` parameter, you can declare an associative array of search terms (keys) and the string into which those must be converted (values), ensuring a successful search.

Finally the `query_builder_processor` parameter can host a function, allowing to process the query built in the query builder in any way not supported by the default parameters or settings. The function takes the query as input string, and gives the modified query back as output string.

Table-body

Some basic cell configuration

If a cell is just supposed to be displayed, no configuration is needed. It will be displayed, that's all.

If you need a cell to be editable, that needs to be set:

```
tablename: {

    cellname: {
        editable: true
    }
}
```

Note that editing a cell value is usually only possible when the database table has a primary key. Lex'it will detect that automatically and will be able to update the right row/cell.

If the database table doesn't have a primary key, and it's for some reason not possible to assign any, you'll have to design a custom update function, telling Lex'it which cells must be taken as a row-reference: without it, Lex'it just can't know where to operate!

```
tablename: {

    cellname: {
        editable: true,
        editfunc: function(t, nRow, value){

            // let's say we can point to a row by referring to
            // two different fields (kind of composite key)

            var sRef1 = fn.getDataFromSiblingNode(nRow, "column1");
            var sRef2 = fn.getDataFromSiblingNode(nRow, "column2");

            // do the update using those references:

            fn.updateTableGivenFieldValues(t,
                {"column1": sRef1, "column2": sRef2}, // point to this row!
                {"cellname": value},   // assign value entered by user
            );
        }
    }
}
```

There might be other reasons for designing custom updates like this, for example because you just need all kinds of custom stuff to happen. For example you want the entered value to be modified in some way before sending it to the database. In this example, we assume that the table does have a primary key, so we'll be updating given a row node:

```
tablename: {

    cellname: {
        editable: true,
        editfunc: function(t, nRow, value){

            // do something with the value entered by the user

            value = value.replace( … );

            // do the update programmatically:

            fn.updateTableeGivenANode(nRow, // point to the row
                {"cellname": value},    // assign the (modified) value
            );
        }
    }
}
```

Note that this can be achieved with less code, since Lex'it does have a '`editpreprocess`' parameter. So the following code should do exactly the same thing:

```
tablename: {

    cellname: {
        editable: true,
        editpreprocess: function(value){

            return value.replace( … );

        }
    }
}
```

Set a wysiwyg editor onto a cell

Lex'it can set a wysiwyg editor to be opened upon clicking a cell. Just use the following declaration. Beware: this should NOT be used in combination with the 'editable' parameter:

```
tablename: {

    cellname: {
        editor: true,
        editor_config: {}, // use the trumbowyg parameters
        edit_callback: function(t, n){
            ...
        }
    }
}
```

Dealing with rows

Get the first row

```
var nRow = fn.getFirstRowNodeFrom(t);
fn.message("OK", "The very first row has ID " + nRow.id );
```

Which row is currently active?

The active row (the row being highlighted in the GUI when going up/down with the arrow keys) can be retrieved with this function:

```
var nRow = fn.getActiveRowNode(t);
fn.message("OK", "You're just clicked row ID " + nRow.id );
```

Get the row before/after another row

```
var nPrecedingRow = fn.getPreviousRowNode(nAnyRow);

var nNextRow = fn. getNextRowNode(nAnyRow);
```

Which row is now selected? At which index?

```
var nRow = fn.getFirstSelectedRowNodeFrom(t);
fn.message("OK", "The ID of the currently selected row is " + nRow.id );

var iRowNumber = fn.getIndexOfFirstSelectedRowNodeFrom(t);
fn.message("OK", "Row number " + iRowNumber + " is currently selected";
```

How many rows are now selected?

```
var iNumberOfSelectedRows = fn.getNumberOfSelectedRowNodes(t);
```

How many rows are now visible on screen?

Two kind of values need to be distinguished here:

- The display length, which is the max number of rows allowed to be shown at once (of course this value plays a role in paging as well)
- The number of visible rows, which can match the display length, but not necessarily, because the result set of a query might return less rows than the display length allows.

```
var iDisplayLength = fn.getCurrentDisplayLength(t);

var iNumberOfVisibleRows = fn.getNumberOfVisibleRows(t);
```

Iterating over rows

This can be done two ways, depending on the type of object:

```
// iterating over row nodes
$(aRows).each(function(){
	…
});

// iterating over rows in a DataTables instance
oRows.every(function(){
	…
});
```

Get number of rows

```
// number of rows in whole table
var iTotalInDatabase = fn.getTotalNumberOfRows(t);

// number of rows visible on screen
var iTotalOnScreen = fn.getNumberOfVisibleRows(t);

// number of rows after filtering
var iFilteredRows = fn.getNumberOfFilteredRows(t);
```

Get the index of a row (row-number)

```
// get the (0-based) number of a row node on the screen, that is: the row number
within current display range
var iRowNr = fn.getRowNodeNumberOnScreen(nRow);

// get the number of a row node (0-based), that is: its index within the set of
ALL database rows
var iRowNr = fn.getRowNodeIndex(nRow);
```

Get all visible rows

You can get all visible rows of a table (so as to perform some operation with those) by using `fn.getAllRowNodes(t)`:

```
var aRows = fn.getAllRowNodes(t);
var aApprovedRows = new Array();

$(aRows).each(function(){

	var thisRow = this;

	var bApproved = fn.getDataFromCellInRowNode(thisRow, "approved");
	var sId = fn.getDataFromCellInRowNode(thisRow, "id");

	if ( sf.isCheckboxTrueValue(bApproved)){
		aApprovedRows.push(sId);
	}
}
```

Get all visible rows meeting some filters

You can select all rows, given some values expected in some given columns. The result is an array that can be used the same way as the result set of `fn.getAllRowNodes(t)`:

```
var aRows = fn.getAllRowNodesWhere(t, {"col1": value1, "col2": value2});

$(aRows).each(function(){

       // do something
}
```

Select or unselect row programmatically

Rows can be selected or unselected one at the time…

```
// given a row node
var nRow = fn.getRowNodeWhere(sTableName, {"id": 123};
fn.selectRowNode(nRow);
fn.unselectRowNode(nRow);

// or given a table and a row number
fn.selectRowNode(sTableName, iRowNumber);
fn.unselectRowNode(sTableName, iRowNumber);
```

…or all at once:

```
fn.unselectAllRowNodes(sTableName);
```

Refresh the row data

In Lex'it, a row can be refreshed the same way a table can:

```
// refresh a table: t must be a table name or an API object instance
fn.refreshTable(t, fnCallback);

// refresh a row: n must be a row node (or a cell node in a row)
fn.refreshRow(n, fnCallback);
```

Dealing with cells (columns)

Get a cell and its type, given a row node and a column name.

```
var nCell = fn.getCellInRowNode(nRow, sColumnName);

var sCellType = fn.getCellNodeType(nRow, sColumnName);
// this function returns 'text', 'checkbox', 'selectbox' or 'unknown'
```

When dealing with an ENUM type or so, you can retrieve the allowed values of a column.
Beware: this works in two cases:
[1] when the allowed values of the column are declared with the `'choosefrom'` parameter, or
[2] when the column has an ENUM type (that is, the allowed values were declared in the database).

```
var aAllowedVals = fn.getAllowedValuesOfColumn(sTableName, sColumnName);
```

Check if a column is visible, etc.

Much of the current state of a table is stored in a kind of registry, which can be access through the `mt` (multitables) namespace (see **Tips and Tricks**, section **Lex'it internal registry**).
For example, the registry keeps track of the columns currently visible:

```
var aVisibleColumns = mt.getListOfVisibleColumnsOf(sTableName);
var bColumnIsVisible = aVisibleColumns.includes(sColumnName);
console.log(sColumnName + ' is ' + (bColumnIsVisible ? 'visible': 'not visible');
```

Assign a mouse event to a cell

To assign a mouse event (`click, dblclick, mouseenter, mouseleave`, …) to a cell, just use it as a parameter name in the cell configuration:

```
"tablename": {
	"columnname": {

		"click": function(t, n, e){
			// do something
		}
	}
}
```

Assign a mouse event to an element in a cell

Let's imagine we've got cells containing not only text data, but also some clickable icons for calling some functions. If a cell only contains an icon, there is no problem at all: the icon click event will be caught as a cell click event, since the icon is part of a cell.

But if a cell contains both text AND some icon at the same time, we do have a challenge. Since Lex'it attaches mouse event to cells, it is primarily impossible to assign a function to only *a part* of a cell, like a clickable icon from our example.

One way to solve this, is retrieving the true target of the click event, as soon as it is caught as a cell click event. The event variable can be retrieved as a 3rd function parameter, after the usual parameter `t` and `n`:

```
"tablename": {

	"columnname": {
		"click": function(t, n, e){ … }
	}
}
```

Let's say our icon is implemented in our text cells as button with classname 'hello'. We can verify if the icon was clicked the following way:

```
"tablename": {

	"columnname": {

		"click": function(t, n, e){

			var bIconClicked = $(e.target).is("button") &&
				$(e.target).hasClass("hello");

			if (bIconClicked){
				console.log("the icon was clicked!");
			}
	}
}
```

Highlighting data in cells

Selected parts of the cell textual content can be highlighted. This is especially handy to show that some text has been selected or so.

Let's say some words were selected in a cell. The onsets and offsets of the selected words can easily be retrieved (see section **Read data** / **Read selected text in cell**).
The onsets/offsets must be put into an array, and given to the `fn.getHighlight` function as a parameter so as to highlight the words:

```
tablename: {

    columnname: {

        "click": function(t, n, e){

              // get the full text content of the cell
              var sText = fn.getDataFromSiblingNode(nRow, sColumnName);

              // get the selected text
              var oSelection = fn.getSelectedTextInNode(n);

              // extract the onset/offset of the selected text
              var iStart = oSelection.start;
              var iEnd = oSelection.end;
              var aOnsetsOffsets = [ [iStart, iEnd] ];

              // apply highlighting between this onset/offset
              var sHighlightText = fn.getHighlight(sText, aOnsetsOffsets, sColor);

              // put the highlighted text back into the cell
              fn.putDataIntoCellNode(nRow, sColumnName, sHighlightText);
       }
    }
}
```

Highlighting can be removed too:

```
// get the text content of a cell
var sText = fn.getDataFromSiblingNode(nRow, sColumnName);

// remove the highlighting
var sCleanText = fn.removeHighlight(sText);

// put the cleaned up text back into the cell
fn.putDataIntoCellNode(nRow, sColumnName, sCleanText);
```

Sorting data

A table is normally sorted according to the `column_sorting` parameter in the configuration file.
But it is possible to change the way a table is sorted programmatically afterwards, that is, at runtime:

```
fn.setSorting(sTablename, {"column1": "asc", "column2": "desc", etc.});
```

And it is possible to retrieve information about the way a table is sorted as well.
Here is how to get the sorting columns of a table, in priority order:

```
var aColumnNames = fn.getSortingColumns(sTablename);
```

And the sorting direction of the columns, in the same order:

```
var aSortDir = fn.getSortingDirections(sTablename);
```

Note: the way data is sorted depends on the collation set in the database. If needed, one can have a project use a collation different from the default one. To do so, use the 'collation' parameter in the `.database` configuration file (see section **The .database file**).

Read data

Read text from cells, columns, or rows

Given a cell node or a row node, you can read data from that node or its siblings by calling the following functions:

```
// get data from a cell, given a cell node
var sData = fn.getDataFromCellNode(nCell);

// get data from a cell, given its name and the row node it belongs to
var sData = fn.getDataFromCellInRowNode(nRow, sColumnName);

// get data from a cell, given its name and the node of another cell
// (this is typically the type of function to call when you have
// clicked some cell and you want to know the corresponding row ID, which
// is stored in another column of the same row)
var sData = fn.getDataFromSiblingNode(nCell, sOtherColumnName);
```

A particular case is that of checkboxes. Some function makes sure their value is read in a reliable way:

```
var bTrueOrFalse = fn.getCheckboxValue(nCell);
```

It is also possible to get the data of all cells of a column, by specifying its name:

```
var aData = fn.getDataFromColumn(sSomeTable, sColumnName);
```

The functions here above are about reading data from screen. But you can also read a row straight from the database without displaying it. The result is stored into an associative array. Which function to use depends on the data at your disposal (a row ID or some other field value to match):

```
var oRow = fn.getRecord(sSomeTablename, sRecordId, fnCallback, fnErrorHandler);

var oRow = fn.getRecordGivenFieldValues(sSomeTablename, {"column": valuetomatch},
fnCallback, fnErrorHandler);
```

Read selected text in cell

If you select some text in a cell (by highlighting it just like in a Word document or such), that can be detected. Typically, this has to be implemented as a click event on a column. For example:

```
tablename: {

    columnname: {

        "click": function(t, n, e){

              var oSelection = fn.getSelectedTextInNode(n);

              var sText = oSelection.text;
              var iStart = oSelection.start;
              var iEnd = oSelection.end;

              fn.message("Info", "You selected the text "+sText+" starting
                at index "+iStart+" and finishing at index "+iEnd);
       }
    }
}
```

A special case is selecting by clicking onto a word. In that can the word might not be highlighted, but Lex'it can automatically search for the edges (begin, end) of the word clicked upon and return its full string:

```
tablename: {

    columnname: {

        "click": function(t, n, e){

              var oSelection = fn.getWordClickedUponInNode(n);

              var sWord = oSelection.text;
              var iStart = oSelection.start;
              var iEnd = oSelection.end;

              fn.message("Info", "You clicked the word "+sWord+" starting
                at index "+iStart+" and finishing at index "+iEnd);
       }
    }
}
```

Read IDs of rows

IDs of rows on screen can be read by calling this function:

```
fn.getIdFromTable(sSomeTablename, {"column": valuetomatch}, fnCallback,
fnErrorHandler);
```

Dealing with Booleans

Reading Booleans can be confusing as, depending on the database version, data type, etc. the true or false values of Boolean turn out to be different, like `t/f` or `1/0`.
We have a set of functions to deal with that (possible) issue:

```
var bValueIsTrue = sf.isCheckboxTrueValue(someValue);

var bValueIsFalse = sf.isCheckboxFalseValue(someValue);


// even easier:

var bTrueBooleanValue = lexutil.translateBoolean(mSomeValue);
```

Write data

Write data in editable cells

There are two main different ways of writing data into cells.

- The first one is about putting data into screen cells, without modifying the underlying database. Such an operation can be convenient for modifying the text format (e.g. adding some style for readability) or doing any other type of text pre-edition before validation (that is, before actually putting the data into the database).
- The second one is about putting data into the database, without necessarily modifying the screen cells. If the screen cells are visible at the time of writing into the database, you can reflect the changes by refreshing the table (`fn.refreshTable`).

Putting some content into screen cells is taken care of by this function:

```
fn.putDataIntoCellNode(nRow, sColumnName, sContent);
```

For checkboxes, some special functions are needed:

```
fn.toggleCheckbox(nRow, sColumnName, fnCallback);

fn.uncheckCheckboxes(nMixed, aListOfColumns, fnCallback);
```

As far as select boxes are concerned (ENUM type or so), good practice would be to check for the allowed values before actually editing the cell, so as to prevent illegal data. Here is an example:

```
tablename: {

     cellname: {

          "click": function(t, n, e){

               // check for allowed values for the selectbox
               var aAllowedVals = fn.getAllowedValuesOfColumn(t, cellname);

               // show dialog with those values to choose from
               fn.prompt("Select value please", ["your choice"], [aAllowedVals],

                    // process that
                    function(resp){
                         var sSelectedValue = resp["your choice"];
                         // do something cool now
                    });

          }
     }
}
```

Sending data to the database

Writing into the database is taken care of by the following set of functions. Here is how to insert data. Only two sets of parameters are compulsory: a table name, and data to insert.

```
fn.insertIntoTable( sSomeTablename,
    {"column1": newValue1, "column2": newValue2);
```

If you need to retrieve the ID of the inserted row, just specify the name of the column holding such IDs. The ID will can be retrieved through the callback function parameter:

```
fn.insertIntoTable( sSomeTablename,
    {"column1": newValue1, "column2": newValue2},

    "row_id",   // name of the ID column in this table  (otherwise: null)

    function(idOfNewRow){
        fn.message("OK", "You just added row ID "+ idOfNewRow);
    });
```

As a final parameter, you can add an error handler too (see the **Error handlers** section about that):

```
fn.insertIntoTable( sSomeTablename,
    {"column1": newValue1, "column2": newValue2},

    null,   // null means no ID needs to be returned

    function(resp){
        // do something in the callback
    },
    function(errorInfo){
        // do something in the error handler
    });
```

Updating a table can be done using two different functions. Which one to use depends on the reference data you have at your disposal: a node, or a table name and some field values to match in it.

```
// update this row (node)
fn.updateTableGivenANode( nMixed,                                    // match this
    {"column1": newValue1, "column2": newValue2},                    // update
this
    fnCallback, fnErrorHandler);

// update the row matching these field values
fn.updateTableGivenFieldValues( sSomeTablename,
    {"someColumn": valuetomatch, "otherColumn": otherValueToMatch}, // match this
    {"column1": newValue1, "column2": newValue2},                    // update
this
    fnCallback, fnErrorHandler);
```

Context menus in cells or rows

Context menus can be set on any cells of a table. So do so, two things need to be set: a list of menu items, and a callback which will perform actions assigned to the items of the context menu items.

Here is a simple example: we set a context menu on column/cell 'lemma'. When clicked upon with the mouse context menu button, the user is given a menu with two items to choose from:
[1] show the paradigm of the lemma or [2] open the dictionary article of that lemma

```
"table_name": {

       "lemma": {
              "contextmenu": {
                     // here the structure of the context menu is given
                     "items": {
                            // here we set keys to which user friendly names
                            // are assigned
                            "wordform":  {"name": "Show wordforms"},
                            "dict":      {"name": "Open dictionary"}
                     },

                     // callback function called after the user has chosen
                     // an option in the context menu
                     "callback": function(table, node, key, options){
                            if (key == 'wordforms')
                            {
                            var sLemId =
                              fn.getDataFromCellInRowNode(table, node, "lemma_id");

                            fn.callTable("wordforms", {"lemma_id": sLemId });
                            }
                            else if (key == 'dict')
                            {
                            var pid =
                              fn.getDataFromCellInRowNode(table, node, "pid");
                            var url = "http://dictionary.url?pid=" + pid;
                            window.open(url);
                            }
                     },

                     // style to apply if needed
                     "class": "…"
              }
       }
}
```

Note: if the "name" of an item needs some html (like "<B>" etc.), make sure to add `"isHtmlName": true`, like this:

```
                     "items": {
                            // here we set keys to which user friendly names
                            // are assigned
                            "wordform":  {"name": "Show <B>wordforms</B>",
                                          "isHtmlName": true}
                     }
```

Note: context menus in Lex'it are implemented with the plugin 'jQuery contextMenu'. Items can be defined following the native API of that plugin, which allows for more than is described here (https://swisnl.github.io/jQuery-contextMenu/docs.html).

Refreshing a table, or only single rows

If you want to make sure that the screen rows reflect the current content of the database, you can call the `fn.refreshTable()` function. For sake of speed, Lex'it won't necessarily return an exact table count: when dealing with very large tables, an approximate table count is much faster to get, and mostly good enough for paging aims.
In cases in which an exact count is required, do the following:

```
// tell Lex'it to do an exact count of the number of rows of the table
fn.forceExactCount();

// refresh the table
fn.refreshTable(t);
```

Instead of refreshing the whole table view, you can choose to refresh a single row only. For example when you expect only that row to have changed. The advantage of refreshing only one row (instead of the whole table) is obviously speed, since sorting and such don't apply to a single row:

```
// refresh one particular row
// only the columns specified in aColumnsToUpdate will be refreshed on screen
fn.callRecord(nRow, aColumnsToUpdate, fnCallback, fnErrorHandler);
```

Remove data

Remove a row

A row can be removed by calling one of the following functions, depending on the data at one's disposal: a row, or a table name and some field values to match in it:

```
// delete this row node
fn.removeFromTableGivenANode( nRow,
    fnCallback, fnErrorHandler);

// delete the row matching these field values
fn.removeFromTableGivenFieldValues(sSomeTablename,
    {"column1": value1, "column2": value2},
    fnCallback, fnErrorHandler);
```

Implement an autocomplete in an editable cell

When you want to get suggestions when typing some data in a cell, an autocomplete function has to be implemented. First we need a database function to provide the data to choose from:

```
CREATE OR REPLACE FUNCTION api.get_lemmata(text)
  RETURNS text AS
$BODY$
    SELECT string_agg( modern_lemma || '/' || lemma_gigpos || ':' || lemma_id ,
'|') -- output (=suggestions) must be pipe separated
    FROM (
        SELECT modern_lemma, lemma_gigpos, lemma_id
        FROM lemmata
        WHERE modern_lemma ~* ('^'||$1)
        ORDER BY modern_lemma
        LIMIT 10
    ) x;
$BODY$
  LANGUAGE sql VOLATILE
  COST 100;
```

When we've got that, implementing the autocomplete is easy:

```
fn.setAutoComplete("someTable", "correction", false, "api.get_lemmata");
```

This will implement an autocomplete in table 'someTable', column 'correction', and the suggestions will be provided by the database function '`api.get_lemmata`'.

Beware: some functions might issue other separators. Those can be specified with two additional parameters, `sPrimarySeparator` and `sSecondarySeparatoropt`:

```
fn.setAutoComplete("someTable", "correction", false, "api.get_lemmata",
sPrimarySeparator, sSecondarySeparator);
```

By default, the autocomplete expects two things before getting active:
* it requires an input length of 2 chars
* and it will wait 750 milliseconds after the last key strike (which allows the user to type several chars without triggering the autocomplete, as it will start working only after the delay following the last key strike)

The default values of these parameters can be modified thanks to two extra function parameters, `iMinLength` and `iDelay`:

```
fn.setAutoComplete("someTable", "correction", false, "api.get_lemmata",
sPrimarySeparator, sSecondarySeparator, iMinLength, iDelay);
```

Wildcards in columns configuration

If you want to assign some setting to several columns at once, you can use wildcards. If the setting must NOT apply to some columns, just add explicit configuration for those columns: explicit configuration will win over wildcards!

```
"*": {                      // all columns
     parameter: value
},

".*_id$": {                 // all columns whose name end with 'id'
     parameter: value
},

"^modern": { {              // all columns whose name begin with 'modern'

     parameter: value
},


"some_column": {
     parameter: value
}
```

Note: this applies to table columns configuration, but not to forms fields.

Navigate programmatically

You can go to the previous or the next page of a table by calling these functions:

```
fn.goToPreviousPage(sSomeTable);

fn.goToNextPage(sSomeTable) ;
```

Navigating to a particular page, given some cell value to be found on that page, is done by:

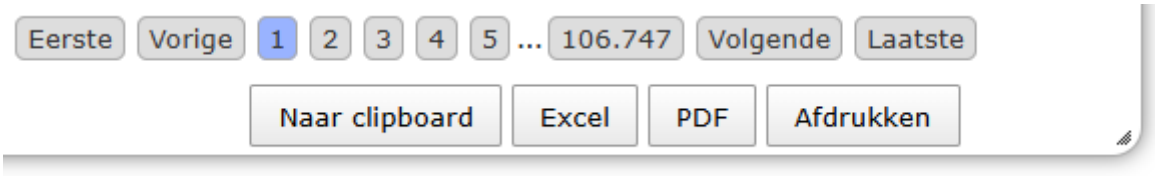
```
fn.goToTheRightPage(sSomeTable, sColumnName, sColumnValue);
```

If you need this function to trigger some callback after completion, you should use the `fn.addDrawCallback()` function, which is recommended for any function triggering a draw event but lacking a callback function parameter:

```
// set the callback in advance
fn.addDrawCallback(t, function(){
       // do something
});

// call the function, and the callback will be called afterwards
fn.goToTheRightPage(t, sColumnName, sColumnValue);
```

Table footer



Pagination buttons

In Lex’it tables, pagination buttons are displayed by default both in the header as in the footer of the tables. If you don’t need such buttons in the footer, set the following:

```
  oTableSettingsList = {

   "your_table": {
      "pagination_at_bottom": false
   }
};
```

Compact footer

By default, the export buttons are displayed under the pagination buttons, at the bottom right corner of the table body. This layout, which is inherited from the original DataTables style, might make the table a bigger/taller than necessary. This can solved, by rendering the export buttons in a more compact way: left aligned, on the same line as the pagination buttons. To do so, just set this:

```
oTableSettingsList = {

   "your_table": {
      "compact_footer": true
   }
};
```

Export buttons visibility

The export buttons are available by default. If those are not needed in your project, you can hide them by setting the following parameter:

```
oTableSettingsList = {

   "your_table": {
      "export_buttons": false
   }
};
```

Add a full export button

The default export buttons of Lex’it are by default ‘screen export buttons’. That is: the export buttons only output the table rows that are displayed on screen when the export is started.
This is by design: by giving only limited export capabilities, we avoid overloading the database engine with sudden massive exports, and we also prevent the database from being fully exported while we might not want that to happen!

But, in your own project, you might need a full export function. That can be made available with a bit of configuration:

```
oTableSettingsList = {

   "your_table": {

       // we want a full export button for this table:

       "full_export_button": {

              // set export button label

              "nice_name": "Volledige Excel Export",

              // where do we put it, within the default export buttons
              // available options: “first” or “last”

              "position": "last",

              // set the buttons groups labels,
              // that is, one label for the default ‘screen export’ buttons
              //  and one label for the ‘full export’ button
              "buttons_groups_labels": ["Scherm:", "Filter:"]
       }
   }
};
```

This configuration will allow the following layout:



Callbacks

Table callbacks

Typically, table callbacks have to be set in the `oTableSettingsList`.

For several stages of the tables rendering, you can set callbacks. For example, let's say you want to change the color of some cells or rows when those meet some constraint. You will have to declare a callback function in the following way:

```
tablename: {

	"callback": function(t){
		var oRows = fx.getAllRows(t);
		oRows.every(function(){
			var nCellSelector =
				$( fx.getNode(oThisRow) ).find("td");
			if (some constraint) {
				nCellSelector.find("input").css("color",
			"green");
			}
		});
	},
	"repeat_callback": true
}
```

The `repeat_callback` part makes sure that the callback is not called only once (say: at table initialization) but at every new table draw event. Of course, leaving out the `repeat_callback` part, or explicitly setting it to `false`, is the way to set a initialization callback, as that should only be called once.

Next to the draw callback, we have callbacks targeting other events like clicking the reset or close button: `prereset_callback`, `close_callback`. Those callbacks must be declared the same way as above (though the `repeat_callback` part won't be needed there):

```
tablename: {
	"prereset_callback": function(t){
		// do something upon reset, but before the table is re-loaded
	}
}
```

A callback can also be called when you close a table:

```
tablename: {
	"close_callback": function(t){
		// do something when the close button is clicked
	}
}
```

You can also set a pre-initialization callback. This can be used to carry out any task before the table is loaded at all. Note that this `preinit_callback` is different from the `prereset_callback`: the first one is executed the very first time a table is called, before that table is even loaded or built in the GUI, whereas the second one is about carrying out some task when the reset button is clicked, which can only happen when the table was loaded already.

```
tablename: {
	"preinit_callback": function(t){
		// do something before the table is loaded
	}
}
```

Finally, we might want to trigger a callback upon table destruction (which happens when a table is removed from the screen):

```
tablename: {
	"destroy_callback": function(t){
		// do something as soon as the table is removed from screen
	}
}
```

Function callbacks

Lots of Lex'it functions allow a callback to be given as an argument. That way, Lex'it will wait till the function is fully executed before executing the code given as a callback.

```
fn.callTable(t, …, function(){
	// some code for your callback
});
```

Sadly, for multiple reasons, some functions lack the possibility to give a callback as an argument. But in many cases, we have a workaround: set a so-called 'draw callback'. That is a function to be called as soon as the table is redrawn, for example when browsing the table or refreshing it.

The following declares a draw callback for table `t`. Note that this works only once. As soon as this callback was called once, it won't be called again.

```
fn.addDrawCallback(t, function(){
	// do something here
});
```

Error handlers

Error handlers can be set in different ways: in the configuration of a column, or as a function parameter:

Table columns error handlers

When a column is `editable`, just add a `editerrorhandler` key with some handler as a value:

```
      "editable": true,
      "editerrorhandler": function(jqXHR, textStatus, errorThrown){
            fn.message("Error!", "Something went wrong! More info:"+
               JSON.stringify( err["jqXHR"] ) )
             );
            },
```

Function error handlers

Quite some functions allow for a custom error handler to be declared as an argument.

The error handler is to be called with one argument:

```
fn.callFunction(sFunctionName, […], function(response){…},
function(errorInfo){…});
```

…where `errorInfo` is an object containing at least the following keys, giving access to extra info in the corresponding values:

```
errorInfo = {
  "jqXHR": …, "textStatus": …, "errorThrown": …,
  "lexit_function": "fn.callFunction"
}
```

The "`lexit_function`" key indicates which function caused the error.

Show a clean error message, when the database threw an exception

The SQL database can throw an exception for various reasons. One can be that some operation performed in the Lex'it GUI caused a database integrity issue (like constraint violation or so). Another reason can be, that some function raises an exception when some unwanted situation occurs, as defined in the function body.

In both cases, the exception will cause a malfunction in the Lex'it webservice, leading the Lex'it GUI to receive a HTTP 500 error, which will be displayed in a dialog.
Though informative for a developer, such a dialog is not very helpful for the average Lex'it user. She or he will only need a clear message telling what she/he did wrong. To achieve that, the most logical solution is to generate an error message at the very location where the error occurred: in the SQL function that raised the exception.

That generally looks like this:

```
CREATE OR REPLACE FUNCTION some_function()
    RETURNS trigger
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE NOT LEAKPROOF
AS $BODY$
DECLARE
BEGIN
…

IF (some condition to be met) THEN
       RAISE EXCEPTION 'The thing you tried is not allowed.
              Do that instead.';
END IF;

…
END;
$BODY$;
```

When the exception is raised, the Lex'it GUI shows a dialog with the full server error response (a long HTTP 500 error message, as we already said). This response will contain the exception text issued by the SQL function, but since it's surrounded by a lot of technical details, it won't stand out and may not be noticed by the user.

Solving that is very easy: put the text of the exception between <lexit> tags. When the Lex'it GUI finds those, it cuts that out of the very long server response and only displays that! The result is a clean, readable and relevant error message!

```
…

IF (some condition to be met) THEN
       RAISE EXCEPTION '<lexit>The thing you tried is not allowed.
              Do that instead.</lexit>';
END IF;

…
```

Dialogs

Classic dialogs

Lex'it support several kinds of dialogs:
The classic JavaScript functions `alert()` and `confirm()` have their Lex'it equivalents, provided with some callbacks:

```
fn.message(sTitle, sMessage, fnFunction);

// since 'confirm' can be answered negatively, we need a 'cancel function' too,
// which handles what happens if the user clicks 'no'
fn.confirm(sTitle, sMessage, fnFunction, fnCancelFunction);
```

Entering data in a dialog

Data to be typed in

Lex'it has some more functions meant to prompt the user to enter information.

The first of those functions – `fn.prompt()` – displays a dialog in which the user is asked to fill in some fields. The field labels and corresponding default values (most of the time empty values) must be given as separate arrays.

```
var aFieldNames = ["first name", "last name"];
var aValues = ["", ""];

fn.prompt(sTitle, aFieldNames, aValues, fnFunction, fnCancelFunction, bTextarea,
aColsAndRows)
```

Fields can be of different types. Given the format of the values, Lex'it will adapt automatically:

- if a value is an array, Lex'it will display a select-box
- if a value is a Boolean, Lex'it will display a checkbox
- if a value is a string or a number, Lex'it will display a text field

Some particularities:

- In case of a select-box, a value to be selected by default must have the string `'::selected'` attached.
- To disable a field (allowing the user to see its default value, without being able to change it), its value must have the string `'::disabled'` attached.
- If you want a datepicker, attach the string `'::datepicker'` to the field value.
- If a field must be a textarea, just attach the `'::textarea'` string to its value.
- If a field must be a WYSIWYG editor, just attach the `'::editor'` string to its value.

```
var aFieldNames = ["name", "gender", "employed", "extra_details"];
var aValues = ["", ["male::selected", "female", "other"], true, "::editor"];

fn.prompt(sTitle, aFieldNames, aValues, fnFunction, fnCancelFunction [,
bTextarea, aColsAndRows])
```

Data to be chosen out of a list of items

The second type of function displays a dialog in which the user is asked to select one or more items.

The items to choose from must be given as an array list, and if some value(s) must be pre-selected, that can be specified in another array:

```
var aAllOptions = ["nike", "addidas", "other"];
var aAlreadyChosen = ["other"]; // pre-selected

fn.promptSelect(sTitle, aAllOptions, aAlreadyChosen, fnFunction,
fnCancelFunction, mSelectionMode);
```

We have to pay attention to the `mSelectionMode` parameter. Note: the Hungarian notation prefix `m` indicates it can have various types:

* It can be a Boolean. When true, the dialog will be closed as soon as an item is clicked. When false, the dialog will remain open, allowing a 'multiple choice' selection.

```
fn.promptSelect(sTitle, ["item 1", "item 2"], [],
  function(aChoice){
       // a choice was made, do something with it now
  },
  function(){
       fn.message("OK", "Operation cancelled by the user. ");
  },
  true // click is choosing, as the dialog will close directly upon click!
);
```

* It can be a function. This entails the same behavior as false, and the function is executed as a callback, with the selected options as function parameter.

```
fn.promptSelect(sTitle, ["item 1", "item 2"], [],
  function(aChoice){
       // a choice was made, do something with it now
  },
  function(){
       fn.message("OK", "Operation cancelled by the user. ");
  },
  function(selectedText){
       // do something when the item specified in selectedText
       // is clicked, like giving extra info etc.
  }
);
```

Data to be sorted

The third type of function displays a list of items to be sorted. It comes with help functions to process the response:

```
fn.promptReorder(sTitle, aArrayToResort,

	function(){
		// get the new order set by user
		var aNewOrderIdx = fn.getPromptBoxOrder();

		// apply the new order to the input array (aArrayToResort)
		var aResortedArray = fn.processPromptBoxOrder(aPromptBoxOrder,
			aArrayToResort);

		// do anything with the resorted array here
	},
	fnCancelFunction);
```

But this can be put much shorter:

```
fn.promptReorder(sTitle, aArrayToResort,

	function(aResortedArray){

		// do anything with the resorted array here
	},
	fnCancelFunction);
```

Menu dialog

A special type of dialog is 'askToChoose', which asks the user to make some choice, which will automatically trigger a function assigned to that particular choice:

```
var oOptions = {
	"choice 1": function(){ // do something },
	"choice 2": function(){ // do something else }
};

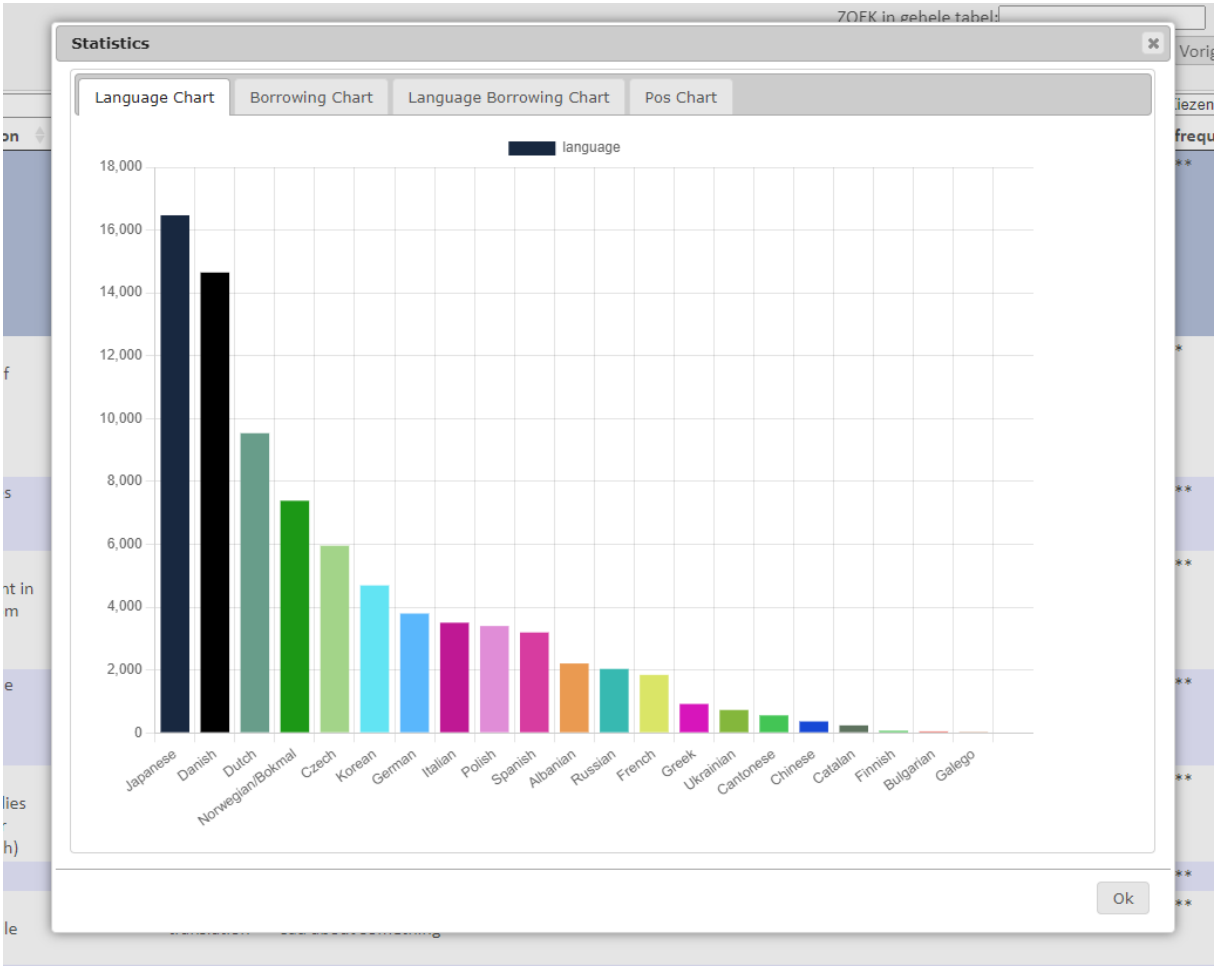
fn.askToChoose(sTitle, sMessage, oOptions);
```

Note that this functionality is also available in `fn.promptSelect(…)`, when using its `mSelectionMode` parameter as a callback function.

```
fn.promptSelect(sTitle, ["choice 1", "choice 2"], [], fnFunction,
fnCancelFunction,
  function(selectedText){
	if (selectedText == "choice 1") {// do something }
	if (selectedText == "choice 2") {// do something else }
  }
);
```

Spread data among tabs in dialog

Let's say you've got several pieces of text, images, charts or whatever to show to the user. Showing that all at once on the screen might be too much. As a solution, you can spread the data among tabs:



This can be achieved by putting the data into an associative array, with the tab names as array keys, and their content as array values.

```
var oTabNames_2_HtmlContent = {

  "Language Chart": "<div class='chart_container' …> …  </div>",
  "Borrowing Chart": "…",
  etc.
};

fn.showTabs(sTitle, sMessage, oTabNames_2_HtmlContent, …);
```

Get rid of the 'OK' button

By default, most dialog functions display an 'OK' button and a 'Cancel' button. The 'Cancel' button is compulsory as you must be capable of canceling some action (or close the dialog). On the contrary, the 'OK' button can be gotten rid of. For example, in a `fn.promptSelect()` call, one of the given options might fulfill the role of an 'OK' button, causing the default 'OK' button to be unnecessary.

Getting rid of the default 'OK' button is easy: just replace the corresponding callback by `null`.

```
fn.promptSelect(sTitle, aAllOptions, aAlreadyChosen,
   NULL,
   function(){…},   // cancel function
   function(){…});  // process the item selection
```

Repositioning a dialog

By default, Lex'it puts dialogs at a central screen position, or – if a table is active – at a position just above or just below the active row (depending on the available room above or under the row). If a dialog appears mispositioned, one can recompute the dialog position by calling the following function:

```
// put the dialog just above or below the active table row
fn.repositionDialog();

// put the dialog at the center of the screen
fn.repositionDialog(true);
```

If needed, you can also move the dialog a bit in one direction or the other:

```
// put the dialog 50px more to the left, and 80px downwards (relative to default
central position)
fn.moveDialog( -50, 80);
```

Closing a dialog programmatically

The dialog currently in front can be closed by calling:

```
fn.closeDialog();
```

Which dialog is opened makes no difference. Any dialog will be closed by this command.

Show a progress indicator and hide it

Show it with:

```
fn.showProcessingMsg(sTableName);
setTimeout(function(){
       // allow the progress indicator to be shown, before a following
       // heavy task prevents the browser from updating the DOM
       fn.callFunction(…);
}, 100);
```

After the job is done, remove the progress indicator by calling this:

```
fn.removeProcessingMsg(sTableName);
```

Key events

Assign event to some key

Any table can be assigned functions to be performed when some keys are pressed. Such functions must be declared in the `oTableSettingsList` array:

```
oTableSettingsList = {

	mytable: {  // the following will work only when this table is active

		"keyup": {

			"a": function(t, e){
				// do something when 'a' is pressed
			}
		}
	}

};
```

In some circumstances, the key events need to be suppressed, for example when some editable cell is being edited while the key event is supposed to operate in completely different circumstances. In such case, you can use the `editFunctionIsActiveNow()` function:

```
oTableSettingsList = {

	mytable: {

		"keyup": {

			"b": function(t, e){
				If (editFunctionIsActiveNow() ) {

					// do nothing, because some cell is edited now!
				}
				else {
					// carry on: do something because 'b' was pressed
				}

			}
		}
	}

};
```

Trigger a key event programmatically

We can trigger some pre-defined key event. Only the first argument is compulsory.

```
Fn.pressKey(sKeyName, nElement, nSubElement);
// or
fn.triggerKeyStrikeOnElement (iKeyCode, nElement, nSubElement);
```

Detect which key was struck

First, you can test if a particular key was pressed:

```
if ( kf.isPressed("alt") ){
       // do something
}
```

But Lex’it can also tell which key was pressed, like:

```
var sPressedKey = kf._getPressedKey();

fn.message("Info", sPressedKey + " was pressed! ");
```

When a key is physically released, Lex’it is expected to detect it. If it fails to do so, this can be corrected programmatically:

```
kf._setPressedKey('released');
```

Calling external resources:

Call a database function and get its output

```
fn.callFunction(sFunctionName, aFunctionArguments, fnCallback, fnErrorHandler);
```

Calling a database function is done by specifying the name of that function and also the database schema it is part of, separated by a dot just like in PostgreSql.

The function output (if relevant), which is put in an associative array, can be retrieved within the function callback (because of asynchronicity), by calling the array key named after the function (but without the dot and the schema name):

```
some_table : {

  some_column : {
                "click": function(t, n, e){

                    var sId = fn.getDataFromSiblingNode(t, n, "id");

                    // call a function and use its output
                    fn.callFunction("api.get_lemma", [ fn.quote(sId) ],

                        function(response){
                           var sLemma = response["get_lemma"]
                           ...
                    });
                }
            }
```

The example here above shows how to retrieve function output consisting of one single value. But some PostgreSql functions might issue sets of keys and values (see section **PSQL functions in Lex'it**). In that case, you can retrieve the value of each key (or: table column) by the array key named after it:

```
                        function(response){
                           var sLemma = response["lemma"];
                           var sPartOfSpeech = response["part_of_speech"];
                           var sId = response["id"];
                           ...
```

Call a webservice

For calling a webservice, only a few parameters are needed, a URL and some parameters, and a callback so as to process the service's response.

```
fn.callService(sURL, aParameters, null, null, function(resp){

	// process response here
});
```

But more parameters can be given, most of which (`sMethod, sResponseDataType, fnCallback, fnErrorHandler`) are quite self-explanatory.

The `oExtraParams` parameter might need extra explanation: it is there for additional ajax parameters that aren't part of the default function parameters, like in the following:

```
var oExtraParams  = {
	"contentType": "application/json; charset=UTF-8",
	"processData": false
};

fn.callService(sURL, aParameters, sMethod, sResponseDataType, fnCallback,
oExtraParams, fnErrorHandler);
```

Proxy and crossDomain

When the service to be called isn't on the same host, and cross-domain requests are not allowed, a JavaScript ajax call will trigger a strict-origin-when-cross-origin error. To prevent that, Lex'it sends the request to its own webservice, which will act as a proxy, avoiding the cross-origin error.

The default workflow is: Lex'it automatically detects if the aimed service is on another host, and then sets `"crossDomain":true`, which tells jQuery to force a crossDomain request. But if you do not wish that to happen, do explicitly set `"crossDomain":false` in the `oExtraParams`, and Lex'it will act as a proxy, that it: it will pass the ajax request to its webservice, which will fire the request and return the response.

If you wish to force use of the Lex'it webservice as a proxy, even when the aimed service is on the same host as Lex'it, set `"useLexitService":true` in the `oExtraParams`. That will overrule the `crossDomain` parameter.

Form views

Basic configuration

Although Lex'it is primarily meant to work with tables, it allows the user to see or edit data in a form as well. Such form can be displayed by clicking the 'View mode' button in the table header (which allows to switch between table and form view). Or it can be activated programmatically by setting the `viewtype` in the `extrasettings` parameter of the `fn.callTable()` function:

```
fn.callTable(tablename,
       {"somecolumn": value}, function(){ … },
       {"viewtype": "form"}
);
```

If the configuration of your project doesn't contain any form declaration, Lex'it will generate a default not-editable form view. But it is possible to design your own form. Technically, a Lex'it form consists of a grid on which text fields, checkboxes, buttons etc. can be placed. We call this grid the 'formgrid'.

The following must be declared in the `oTableSettingsList`.

First declare the grid's dimensions in parameter `"definition"`: number of columns by number of rows to divide the form-area into. The column/row-numbers will be used as coordinates to put text fields, buttons, etc. at).
By default, Lex'it will divide the usual table width/height into the number of columns/rows given as a form definition. But if you wish the form to use a larger/smaller area, you can declare it in the `"size"` parameter, which contains the width/height of the form-area, defined in pixels:

```
tablename: {

       "formgrid": {
              "size": [1000, 300],  // form area will be 1000 x 300 pixels
              "definition": [10, 6] // grid will have 10 columns and 6 rows
                                    within the declared form area
                                    so, one cell will have a resolution of
                                    (1000/10)=100px by (300/6)=50px
       }
}
```

You can also set a background color, align the form within the table frame, and enable or disable the search bar:

```
tablename: {

       "formgrid": {

              "bgcolor": "#88BBCA",
              "align": "center", // possible settings: left/center/right
              "searchbar": true/false
       }
}
```

But the most important elements are the text edit fields and buttons that make up the form. Each of those need a "position" (that is, its column/row coordinates in the grid) and a "definition" (width/height, defined as number of columns/rows in the grid). For styling, a "class" can be given a well. If needed, the way the cell content is rendered can be modified (without modifying the underlying data) using the "render" parameter. Note that the cell needs to be called the same name as the column names of the table the form represents:

```
tablename: {

	"formgrid": {
		"size": [1000, 300],
		"definition": [10, 6],
		…

		"cells": {
			"some_table_column": {
				"position": [1, 1],  // put cell at column 1 row 1
				"definition": [2, 1], // make cell 2 columns wide
				"class": "some_css_classname",
				"render": function(sValue, nRow) {
					sValue = doSomething(sValue);
					return sValue;
				}
			},
			"another_table_column": {
				"position": [1, 3],  // put cell at column 1 row 3
				"definition": [3, 1] // make cell 3 columns wide
			}
		}
	}
}
```

Buttons are to be declared the same way. The labels of the buttons have to be given as keys. The associated array of properties must at least contain a "click" property (associating a function to the button) and a position. You can also set colors, but that's optional.

```
tablename: {

	"formgrid": {

		"size": [1000, 300],
		"definition": [10, 6],
		…

		"buttons": {

			// button label
			"click me!": {

				"position": [2, 5], // put button at column 2 row 5

				"click": function(t){ // execute this upon click
					// do something cool here!
				},
				"color": "black",
				"bgcolor": "red",  // red button with black label
				"class": "some_css_classname" // for more styling!
			}
		}
	}
}
```

The forms also contain default buttons, that is, a 'undo' button and a 'save' button. By default, these buttons will be positioned in the bottom left corner of the form. But this set of buttons can be repositioned the same way as anything else in the formgrid:

```
tablename: {

	"formgrid": {

		"buttonsbar_position": [4, 5] // default buttons at row 4 column 5

		…
	}
}
```

As far as cell editability or visibility is concerned, the formgrid uses the table config by default. But if you need a form cell to behave differently than the corresponding table cell, you can set a different editability or visibility in the formgrid:

```
tablename: {

	"formgrid": {

		"cells": {
			"some_table_column": {
				"position": […,…],
				"definition": […,…],

				"editable": true, // this will win over
				                  //        the tableconfig!
				"visible": true,
				"click": function(t, n){ … }
			},
			"some_other_table_column": {

				"editor": true, // use can also use an editor here!
				"editor_config": {}, // use the trumbowyg parameters
			}
		}
	}
}
```

Finally, you can add some text fields to the form (like a title, or any other kind of static text):

```
tablename: {

	"formgrid": {
		…
		 "textblocks": {
			"some title": {
				"position": […,…],
				"definition": […,…],
				"text": "This could be the title of my form"
			}
		}
	}
}
```

Putting cells into groups

If a form contains a lot of cells, it might be convenient to organize those in groups.

First, you must declare the group names, positions and definitions,. If needed, classes can be added too. Group names are placed into a <h3> element by default, but it is possible to choose another group name element instead:

```
tablename: {

	"formgrid": {

		"cellgroups": {
			"some_group_label": {

				"text": "title of the group",
				"htmltag": "div",		// if not set, default is h3
				"class": "some_css_for_groups",
				"position": […,…],
				"definition": […,…],

			}
		},

		 "cells": {
			……
		}
	}
}
```

As a second step, you must declare which cells belong to one group or another:

```
tablename: {

	"formgrid": {

		"cellgroups": {
			"some_group_label": {

				……
			}
		},

		 "cells": {
			"some_cell_name": {

				"cellgroup": "some_group_label",
				"position": […,…],
				"definition": […,…],
				"order": 1,	// use this for FLEX order if relevant


			},
			 "another_cell_name": {
				……
			},

		}


	}
}
```

Such groups can be rendered in an accordion view, like the one implemented in jquery UI. To achieve that, you can define the following callback:

```
tablename: {

	"formgrid": {
		……

	},

	"callback": function(t){

		// oAccordionConfig : configuration for the jquery UI accordion
		// Lex'it default: {animate: 100, heightStyle: 'content'}

		form.activateAccordion(t, oAccordionConfig);

	},

	"repeat_callback": true

}
```

Callbacks

Like tables, forms have specific callbacks. An important one is the '`save_callback`', which holds any function to be called just after a form was saved.

```
tablename: {

	"formgrid": {
		……

		"save_callback": function(t){ … }

	},
```

Tables as part of a form: lists

Next to fields and buttons, a form may contain tables, allowing to display a paradigm, example sentences, etc. We called such tables 'lists'.

Basis configuration

Just like fields and buttons, a list needs a `"position"`, a `"definition"` and even a `"nice_name"`:

```
tablename: {

	"formgrid": {

		"lists": {

			"some_list": {

				"position": [4, 1], // put list at column 4 row 1
				"definition": [10, 8], // make it large enough
				"nice_name": "This is some list",
				"enter_validation": false // default: true

				"table": {
					…
				}
			}
		}
	}
}
```

Just like in regular Lex'it tables, entering data into tables within forms requires validation by pressing 'Enter'. You can get rid of this validation requirement by adding parameter `'enter_validation': false` to the list configuration.[3]

Lists can be organized in groups, in the same way as cells can (check the Form's **Putting cells into groups** section):

```
tablename: {

	"formgrid": {

		"lists": {

			"some_list": {

				"position": [4, 1], // put list at column 4 row 1
				"definition": [10, 8], // make it large enough

				...,

				"cellgroup": "some_group_label"

			}
		}
	}
}
```

[3] However, using this feature is discouraged, since it seems to be buggy in the current Lex'it version.

The "table" parameter is needed to declare the source of the list data, how it has to be sorted, etc. Just like any table, a list can be assigned a "callback" parameter, referring to a function to be called at each draw event. This callback can be declared at list level or at table level, without any functional difference.

```
tablename: {

	"formgrid": {

		"lists": {

			"some_list": {
				"position": …,
				"definition": …,
				"nice_name": …,  // human readable name in GUI!

				"table": {
					"name": "tablename", // SQL table name!
					"callback": function(t){ … },
					"columns_sorting": {"col1": "asc", …},
					"columns_order": ["col1": "col2", …],
					"columns": {
						"col1": {
							"visible": false, …
						},
						"col2": { … }
					}
				}
			}
		}
	}
}
```

The "table" parameter can be used to declare the columns configuration in the list, just like in tables.

```
	"table": {
		"name": "tablename", // SQL table name!
		"callback": function(t){ … },
		"columns_sorting": {"col1": "asc", …},
		"columns": {
			"col1": {
				"editable": true/false,
				"visible": true/false,
				"click": function(sListLabel, nRow){
					…
				},
				…
			},
			"col2": { … }
		}
	}
```

Finally, the list needs to know which data to load given the current state of the form. For example: if a form displays some lemma, we might need its paradigm to be loaded into a list. And if the user browses through the lemmata, the paradigm list will need to synchronize with the lemma (in other words: this list will need to be filled with the paradigm of the last chosen lemma).

In the underlying database, the data to be loaded as a list is usually linked to the rest of the form data by some ID. So, when the form displays a record with a given ID, this should trigger the list to display content corresponding to (that is, linked with) the very same ID.

To achieve that, go to the "cells" declaration of the form, look up the cell that acts as id-column, and add a parameter called "synchronize_with". The value to be assigned to this parameter must be an associative array, associating one (or more) list-labels to the id-column of each list that needs to synchronize with the ID of the record displayed in the form. When declared, that will tell the lists to be filled with records whose IDs match the ID of the record displayed in the form:

```
tablename: {

	"formgrid": {

		"cells": {

			"lemma_id":{

				"position": …,
				"synchronize_with": {"paradigma": "lemma_id"}
			},
			"modern_lemma": {…}

		},

		"lists": {

			"paradigma": {

				"position": …

				"table": {
					"name": "paradigm",
					"columns": {
						"wordform": {…}
					}
				}
			}
		}
	}
}
```

Important note: if you need to implement the "synchronize_with" parameter, but you do not which to display the cell implementing this parameter, just given the "position" parameter the value "hidden":

```
		"cells": {

			"lemma_id":{

				"position": "hidden",
				"synchronize_with": {"paradigma": "lemma_id"}
			}, …
```

A list also comes with a set of buttons, allowing the user to perform some operations:

- at the top of the list : a button to add a row
- at the end of each row: a button to delete the row, and a button to show content connected to the row

Each of those buttons need to be configured, which can be done in the "buttons" parameter:

```
tablename: {
	"formgrid": {

		"lists": {

			"paradigma": {

				"buttons": { … }

				"table": { … }
				}
			}
		}
	}
}
```

The 'add' button

Adding a row opens a dialog, asking the user to enter values for the new row (by default, this dialog is titled "Add a row", but you can give this dialog another name by using the `"title"` parameter). If a new row needs to be connected to other content (like with foreign keys), some IDs might have to be taken over automatically. You must tell which columns to fill with which content. For example, in the following, the "lemma_id" column of the list will get the value stored in the "lemma_id" cell of the form, and the "part_of_speech" column will get the value stored in the "pos" cell of the active row of the "allowed_pos" list:

```
tablename: {
	"formgrid": {

		"lists": {
			"paradigma": {

				"buttons": {
					"add": {
						"title": "Add a lemma", // custom title
						"copy": {
							"lemma_id": { // copy these values
								"form": "lemma_id"
							},
							"part_of_speech": {
								"allowed_pos": "pos"
							}
						}
					}
				},

				"table": { … }
				}
			}
		}
	}
}
```

Instead of relying on the default behavior, it is also possible to write a custom add-function, which is to be declared with the "do" parameter:

```
tablename: {

	"formgrid": {

		"lists": {

			"paradigma": {

				"buttons": {
					"add": {
						"do": function(sListLabel){
						…
						}
					}
				},

				"table": { … }
				}
			}
		}
	}
}
```

Adding a row to the list requires a specialized function called `lists.addRowToList`.
Here is an example showing a dialog for entering data and save the result as a new row in the list:

```
"do": function(sListLabel){

	fn.prompt("Please enter your name",
		["firstname", "lastname"], [],
		function(resp){

			lists.addRowToList(sListLabel, resp, function(){
				fn.message("OK", "The new row was inserted!")
			});
		}
	);
}
```

The 'delete' button

Just as the "add" parameter for the add-button, the delete-button needs a "delete" parameter as part of the "buttons" configuration. Leaving it will cause the delete-button to be left out of the list. If you want it to behave the default way (that is: remove the row with the current ID), just set `"delete": true`.

But if you wish custom behavior instead, just declare a function within the "do" parameter:

```
tablename: {

       "formgrid": {

              "lists": {

                     "paradigma": {

                            "buttons": {
                                   "delete": {
                                          "do": function(sListLabel, nRow){

                     // retrieve the DataTable object the list represents
                     var sListTableId = lists.getTableIdFromNode(nRow);
                     var oTable = lists.getDataTableObjectOf(sListTableId);

                     // now remove the row
                     oTable.row( $(nRow) ).remove().draw();
                                          }
                                   }
                            },
                            "table": { … }
                            }
                     }
              }
       }
}
```

The 'show' button

The show-button, which should be clicked upon to get content connected to a row, must be configured too. Given the row ID, the configuration must tell in which column of another list the row ID must be looked up to retrieve the desired content. For example, if we want to look up example sentences of a lemma of our lemma list, we need to look up the row- ID of the lemma in the "lemma_id" column of the 'examples' list:

```
tablename: {

	"formgrid": {

		"lists": {

			"lemmata": {

				"buttons": {
					"show": {
						"do": {
							"examples": "lemma_id"
						}
					}
				},

				"table": { … }
				}
			}
		}
	}
}
```

Of course you can declare custom behavior instead:

```
"buttons": {

	"show": {
		"do": function(sListLabel, nRow){

			var someId = nRow.id;

			lists.feed("examples", { "some_id": someId });

			}
		}
	}

}
```

Overview of the form API

The first set of functions targets the form and its cells. Since a form can also contains 'lists' (small tables), we'll discuss a set of functions dealing with those further on.

Forms – retrieve configuration

Get the name of the table feeding the form

Just like when working with tables, we might need to know which table the form is representing (in other words: which database table feeds the form).

You can get the name of the table feeding the current form, given:

- any node in the form
- or the form container ID
  (the ID of the form container `div` has format: '`<tablename>_form`')

```
var sFormTableName = form.getFeedingTable(nNode);     // for forms

// which is, in some way, the same as:
var sTableName = fn.getTableName(t);                  // for tables


// retrieval using the form container ID:
var sFormTableName = form.getFeedingTable(formContainerId);
```

Read the cells configuration

Retrieve the configuration of a form cell (just like `conf.getColumnConfig()` in tables):

```
var oCellConfig = form.getColumnConfig(sFormTableName, sCellName);
```

Get a list of the visible form cells:

```
var aVisibility = form.getVisibleColumns(sFormTableName);
```

Get the list of 'nice names' of the form cells, that is: the cell labels to be displayed as a replacement for the default (database) column names:

```
var aNiceNames = form.getColumnsNiceNames(sFormTableName);
```

Retrieve the name of a form cell, given its node:

```
var sCellName = form.getNameOfCell(nCell);
```

Get the ID of the form container

The form container is a `div` element, containing all content of the form: labels, cells, buttons, lists (the form container has an ID with format: '`<tablename>_form`').

You can retrieve the form container ID, given:

- any node in the form
- or the label of any list in the form

```
var formContainerId = lists.getFormContainerId(nNode);

// or

var formContainerId = lists.getFormContainerId(sFormListLabel);
```

Forms - Read, write and refresh data

Read cell data

The form cells show the content of a database row being shown in the form at that moment. But as soon as manual editing of the cells content begins, this content differs from the content stored in the database. Both can be retrieved: the form cells content (possibly edited) and the corresponding database content (not yet overwritten by edition).

```
// get the value visible in the form right now (this value might differ
// from the database value, if the form cell is being edited right now)

var sCell = form.getDataFromCell(sFormTableName, sCellName);

// get the underlying value of a cell (that is, the value hold in the database)

var sCell = form.getDataFromCell(sFormTableName, sCellName, true);
```

Write cell data

Next to manual edition, you can put data into a form cell programmatically. Beware: the following function gives content to a form cell, but does not save it to the database. Just like any cell edition, the added content needs to be saved in order to be stored in the database". This can be done manually using the 'save' button, or programmatically by using the `form.saveForm()` function.

```
form.setDataInCell(sFormTableName, sCellName, sValue);

fn.message("OK", "Click the OK button to save the form!", function(){
       form.saveForm(sFormTableName);
});
```

Refresh the form data

Just like tables, forms can be refreshed, so as to make sure the current database content is rendered. Of course, any edited content will vanish unless it has been saved:

```
// refresh the form
form.refresh(sFormTableName, fnCallback);
```

Refresh the form layout

After resizing the form container, the form elements might need some rescaling. That can be achieved with the following code:

```
// Apply a callback to resizing events in the form container,
// so as to rescale everything in the form
//
// NOTE the "resize_callback" addresses table events, so it must be declared
// in the tables settings (it won't work if put in the "formgrid" configuration)


tablename: {

       "resize_callback": function(t, ui){
              form.updateLayout(t);
       },
       "formgrid ": { … }
}
```

Save or reset a form programmatically

When working in a form, the user save their work by clicking the 'save' button. Or he/she can reset the form (that is, undo the unsaved changes) by clicking the 'undo' button. This can also be taken care of programmatically:

```
var sFormTableName = form.getFeedingTable(nNode);

form.saveForm(sFormTableName); // save

form.resetForm(sFormTableName); // reset
```

Check if an edited form was saved

You can determine programmatically whether the form is in 'unsaved' state or not, by calling the `form.isUnsaved()` function. This function can help prevent a user from closing a form without saving his/her work:

```
tablename: {

       "close_callback": function(t, ui){

              var sFormTableName = fn.getTableName(t);
              var bUnsaved = form.isUnsaved(sFormTableName);

              if (bUnsaved) {
```

```
                        fn.confirm("The form is not saved yet!",
                               "Would you like to save the form before closing it?",
                               function(){
                                      form.saveForm(sFormTableName);
                               },
                               function(){ … }
                        );
                }

        }
}
```

Forms - search boxes

The search boxes of a form are usually dealt with manually by the user. But those can be emptied/reset programmatically:

```
form.resetSearchFields(sFormTableName);
```

Lists – retrieve configuration

A list is a small table rendered as a part of a form. Each list has a label, which is needed to point at it in functions.

Get a list of all list labels

You can get a list of all list labels in a form, given the form's underlying table name:

```
var aAllListLabels = lists.getListOfListLabels(sFormTableName);
```

Get the label of a single list

The label of a particular list can be retrieved in several ways, depending on the data at one's disposal:

You can retrieve the label of a list given its DataTable object:

```
var sListLabel = lists.getLabelFromDataTableObject(oTable);
```

You can retrieve the label of a list given any node inside it (e.g. a cell or row):

```
var sListLabel = lists.getLabelFromNode(nNode);
```

You can retrieve the label of a list given its container ID:
(the ID of the list container `div` has format '`<formContainerID>___<listLabel>_container`')

```
var sListLabel = lists.getLabelFromContainerId(sListContainerId);
```

Get the name of the table feeding the list

Retrieve the feeding table of a list, given any node inside the list:

```
var sTableName = lists.getFeedingTable(sFormListLabel);
```

Retrieve the configuration of a list

Retrieve the configuration of a list, given any node inside the list:

```
var oList = lists.getConfig(nNode);

// this object can be used we access the full list configuration
var sTableName = oList["table"]["name"];
```

Retrieve the height of a list, as set in the configuration:

```
var sHeight = lists.getDisplayHeight(sFormListLabel);
```

Retrieve the sorting settings (object) of a list, as set in the configuration:

```
var oSort = lists.getSortSettings(sFormListLabel);
```

Get the ID of the list container

Retrieve the list container ID, given any node inside the list:
(the ID of the list container `div` has format: '`<formContainerID>___<listLabel>_container`')

```
var sListContainerId = lists.getContainerIdFromNode(nNode);
```

Get the ID of table feeding the list

Retrieve the feeding table ID (= container ID without '`_container`' suffix) of a list, given:

- any node inside the table
- or given the list container ID (the ID of the list container `div` has format: '`<formContainerID>___<listLabel>_container`')

```
var sTableID = lists.getTableIdFromNode(nNode);

var sTableID = lists.getTableIdFromContainerId(sListContainerId);
```

Retrieve the list table ID (that is: container ID without '`_container`' suffix), given its DataTable object.
Or the other way round: retrieve the DataTable object of a list, given its table ID:

```
var sTableID = lists.getTableIdFromDataTableObject(oTable);

var oTable = lists.getDataTableObjectOf(sTableID);
```

Lists - Read, write and refresh data

Read cell data

You can get the value of a cell in a list, given:

- a row node and a cell name
- or a row number and a cell name
- or a cell node

```
// get the value of a cell in row #1
var sCellValue = lists.getDataFromCell(sListLabel, 1, sColName);

// get the value of a cell given a row node
var sCellValue = lists.getDataFromCell(sListLabel, nRow, sColName);

// get the value of a cell given its node
var sCellValue = lists.getDataFromCell(sListLabel, nCell);
```

```

```

You can get the content of all cells of a column at once. This returns an array, of course:

```
var aValues = lists.getDataFromColumn(sListLabel, sColumnName);
```

Write cell data

Put data into a cell of a list, given the list label, a cell node, and the value to set.
Beware: this function puts data into cells, which can be further edited, but for the values to be saved to the database, the form still needs to be saved (manually or programmatically):

```
lists.setDataInCell(sListLabel, nCell, sValue);

// don't forget to save the form afterwards!
form.saveForm(sFormTableName);
```

Insert a row

Add a row to a list, given an array associating column names to values.
Beware: this adds the row in the GUI, but the user still needs to save the form manually for adding the data to the database!

```
var oArray = {'coumn1': value1, 'column2': value2};
lists.addRowToList(sListLabel, oArray, fnCallback);
```

Refresh a list

Just like tables, lists can be refreshed, so as to make sure the current database content is rendered. Of course, any edited content will vanish unless it has been saved:

```
// show progress indicator
lists.showProcessingMsg(sFormListLabel);

// refresh the list
lists.refresh(sFormListLabel, function(){

	// remove the progress indicator
	lists.removeProcessingMsg(sFormListLabel);
});
```

Lists – dealing with cells

Columns consist of cells that share the same properties as other cells in the column. For us, this means that some cell-related functions will be grouped with column functions, and vice versa. So check the **dealing with columns** section as well.

Retrieve the configuration of cells

Retrieve the configuration of a cell (just like `conf.getColumnConfig()` in tables)

```
var oCellConfig = lists.getCellConfig(sListLabel, sColName);
```

Get the width to be applied to a given cell (or in table terms: column), given the "`lists`" object of the form configuration, the label of the list at stake, and the cell/column name:

```
var sWidth = lists.getColumnsWidth(oLists, sListLabel, sColumnName);
```

A special one: get the true (database) name of a cell, given its nice name:

```
var sName = lists.getTrueColumnName(sListLabel, sNiceName);
```

Or, the other way round, give the nice name of a cell, given its true (database) name:

```
var sNiceName = lists.getNiceColumnName(sListLabel, sColumnName);
```

Get cells node

Get a cell (cell node) in a list, given:

- a row node and a cell name
- or a row number and a cell name
- or a cell node and the cell name of a sibling

```
// get a cell from row #1
var nCell = lists.getCell(sListLabel, 1, sColName);

// or given a row node
var nCell = lists.getCell(sListLabel, nRow, sColName);

// or get the sibling of a cell
var nSiblingCell = lists.getSiblingCell(sListLabel, nCell, sColNameOfTheSibling);
```

Lists – dealing with rows

Get the number of visible rows

We can get the number of rows of a list in different ways:

```
var iNumberOfRows = lists.getNumberOfVisibleRows(sListLabel);

// which is quite the same as the table function:
var iNumberOfRows = fn.getNumberOfVisibleRows(t);
```

Get row nodes

Get a row (node) in a list, given:

- an associative array of column names and values to match
- or a row ID (= primary key in the feeding table)

```
var nRow = lists.getRow(sListLabel, sRowId);

var nRow = lists.getRow(sListLabel, {"some_column": value, "other_column":
othervalue, …});
```

Get selected rows

Get the selected rows (nodes) of a list, given the label of a list.
This function can retrieves row nodes, or a DataTable object representing the rows:

```
// default output is an array of nodes
var aRows = lists.getSelectedRows(sListLabel);

// or get a DataTable object of the rows instead
var oTable = lists.getSelectedRows(sListLabel, true);
```

Add or delete a row

Rows can be added or removed from list with the following set of functions. Beware: these functions process rows in the GUI, but the user still needs to save the form manually for updating the database accordingly!

```
var oColumnsAndValues = {"some_column": value, "other_column": othervalue, …};

// add a row with a some content
lists.addRowToList(sListLabel, oColumnsAndValues, fnCallback);

// remove a row with the same content
var nRow = lists.getRow(sListLabel, oColumnsAndValues);
lists.removeRowFromList(sListLabel, nRow, fnCallback);
```

Select a row programmatically

Select a row in a list programmatically, given:

- an associative array of column names and values to match
- or a row ID (which is the primary key in the feeding table)

```
lists.selectRow(sListLabel, sRowId);

lists.selectRow(sListLabel, {"some_column": value, "other_column": othervalue} );
```

Click buttons in a selected row programmatically

Click programmatically on the 'open' icon in the currently selected row of a list:

```
lists.clickOpenInSelectedRow( sListLabel );
```

Lists – dealing with columns

Columns consist of cells that share the same properties as other cells in the column. For us, this means that some cell-related functions will be grouped with column functions, and vice versa. So check the **dealing with cells** section as well.

Get the column names of a list

Get the list of the columns of a list, given the name of the underlying table. The function retrieve all columns, both visible and invisible:

```
var aAllColumns = lists.getAllColumns(sThisListTableName);
```

Get the datatype of a column

When this function has been called, the application also has knowledge of column types and ENUM values, which can be retrieved the following way:

```
var sColumnType = lists.getColumnType(sTableName, sColumnName);

var aAllowedValuesInColumn = lists.getAllowedValuesOfColumn(sTableName,
sColumnName);
```

Get visible columns

Get the list of columns to use (the ones named in the form configuration), given the "`lists`" object of the form configuration, and the label of the list at stake:

```
var aColumnsToUse = lists.getColumnsToUse(oLists, sListLabel);
```

Get the list of columns that are set to be visible in the form configuration, given the "`lists`" object of the form configuration, and the label of the list at stake:

```
var aColumnsToDisplay = lists.getColumnsToDisplay(oLists, sListLabel);

// quite the same, but easier to use:

var aVisibleColumns = lists.getVisibleColumns(sListLabel);
```

Tips and tricks

Lex'it internal registry

Every single piece of information about tables etc. is stored in a dedicated namespace `mt` (which stands for *multitables*). It is often needed to get info from this registry. Here are some examples:

The most important object of all in Lex'it – the DataTable object of some named table – can be retrieved from there:

```
var t = mt.getDataTableObjectOf(someTableName);

// go to page 1 of the results
t.page( 0 );
```

Note that if a table wasn't loaded yet, the internal registry won't hold any information about it. If you need info about a table but don't want to call it in the GUI, it is possible to call the table silently. That way, all the table info is loaded and can be retrieved with the `mt` namespace functions:

```
fn.callTableSilently(someTableName);
```

You can retrieve the names of all the tables displayed on screen, so as to refresh them all:

```
var aTables = mt.getListOfLoadedTables();

for (var i=0; i< aTables.length; i++){
      fn.refreshTable(aTables[i]);
}
```

The same way, you can get a list of all (visible) columns, so as to change their `css` properties, or whatever else:

```
var aListOfAllColumns = mt.getListOfColumnsOf( t );

var aListOfColumns = mt.getListOfVisibleColumnsOf( t );

for (var i=0; i< aListOfColumns.length; i++){

      var sColumnName = aListOfColumns[i];
      var nCell = fn.getCellInRowNode( nCurrentRow, sColumnName);

      $( nCell ).css("color", sSomeColorCode);
}
```

Check the `mt`-namespace in the JS-docs at `github.com/instituutnederlandsetaal/Lexit/tree/master/jsdoc` to find out what can be retrieved from the Lex'it internal registry.

Using wildcards in configuration

As described in earlier section, the `oTableSettingsList` and the `oTableConfigurationList` contain the configuration of tables and their columns:

```
oTableSettingsList = {

	"table_A": {
		"width": "80%"
	},
	"table_B": {
		"width": "80%"
	}
};

oTableConfigurationList = {

	"table_A": {
		"firstname": {
		    "editable": true
		},
		{
		"lastname": {
		     "editable": true,
		     "bgcolor": "blue"
		}
	}
	"table_B": { … }
};
```

The very same configuration can be declared in a shorter way by using wildcards:

```
oTableSettingsList = {

	"table_.*": {
		"width": "80%"
	}
};

oTableConfigurationList = {

	"table_A": {
		".*name$": {
		    "editable": true
		},
		{
		"lastname": {
		     "bgcolor": "blue"
		}
	}
	"table_B": { … }
};
```

Note: wildcards can be used in table configurations, but not in forms configurations. Use of wildcards in forms wouldn't make much sense anyway, since form configurations mostly declare only a few columns (the ones needed on the form).

Updating a table configuration after initialization

Updating a table configuration can be done by calling the `conf.changeTableConfigValue` function. For example, if you want to assign a function to a click event on column 'some_column' in table 'some_table', you should write:

```
conf.changeTableConfigValue("some_table", "some_column",
       "click", function(){
              do something();
       }
);

// reset the table filters afterwards, taking the new setting into account
fn.resetAllFilters("some_table", true);

// see the result
fn.refreshTable("some_table");
```

Some modifications in the configuration require a table to be removed from the screen to be able to update it. Modified columns visibility settings is such a situation. In such a case, you need to destroy the table, modify the settings and call the table again. Schematically:

```
// first destroy the table (on screen)
tb.destroyTable("some_table", function(){

       // modify the configuration
       conf.changeTableConfigValue( … );

       // call the table back
       fn.callTable(("some_table", … )
});
```

Some complications can occur when more tables are open and visible on screen at the same time. That is: destroying a table and calling it again will cause that table to be appended at the bottom of the screen, underneath all other tables. If this table was (before destruction) first on top of the other tables, the users might get confused as the table now reappears at the bottom when it is called again.

To solve this particular problem, you should read the table position before destruction, and re-apply the table position to the table when it is called (and shown) again. Fortunately, it is quite easy to do. Read the position with `fn.getTablePosition` and give the result as a fourth argument to the `fn.callTable` function:

```
tb.destroyTable("some_table", function(){

       // read the table position
       var oPosition = fn.getTablePosition(t);
       conf.changeTableConfigValue( … );

       // apply the table position
       fn.callTable(("some_table", oSomeFilters, function(){}, oPosition );
});
```

At last, it might be convenient to get rid of some configuration value. Changing a value was done by calling `conf.changeTableConfigValue`, as we saw above. Getting rid of a value must happen like this:

```
conf.removeTableConfigValue("some_table", "some_column", "setting_name");
// setting_name is now removed from the configuration and will act default
```

Create a full new table configuration at runtime

This function is needed when you want to call a fully configured table that didn't exist yet at start up (which is why the table was not registered nor configured yet). Such thing can happen when a new table is created at runtime by some other app (e.g. to store some results) and you wish to see the content in Lex'it right away:

```
fn.registerNewTable(sTableName, sTableDescription, sType, sTableComment,
oConfiguration, oSettings);
```

Synonym:

```
fn.declareNewTable(…);
```

Parameter	Data type	Description
`sTableName`	`String`	Name of the new table to register
`sTableDescription`	`String`	table description (visible to user in tables list)
`sType`	`String`	database table type ('base table', 'view')
`sTableComment`	`String`	table comments, which a user can edit by clicking onto the table name (in the table header) in the GUI
`oConfiguration`	`Array`	An associative array, containing the table configuration, in the way this has to be declared in oTableConfigurationList
`oSettings`	`Array`	An associative array, containing the table settings, in the way those have to be declared in oTableSettingsList

As soon as a new table was created, you can rebuild the tables dropdown in the GUI by calling:

```
fn.rebuildTablesMenu();
```

Set the active table programmatically

In Lex'it, the active table is the one that has focus. You can give another table focus, by typing:

```
fn.setActiveTable(sTableName);
```

By doing this, of course, key actions assigned to this table will get active, etc.
(see `focus_at_mouseover` parameter for similar functionality).

Use libraries

Lex'it `config.js`-files can gradually grow quite large, causing such file to be more and more difficult to maintain. An easy way to solve that is (for example) moving some of the project functions to some functions library.

By default, Lex'it expects libraries to be located in the same directory as the `config.js`-files. And the libraries are expected to have the same core name as the `config.js`-file they belong to. For example, if your project is called 'myproject' and you want a function library to be attached to it, you should have two files named like this:

- `myproject.config.js`
- `myproject.library.js`

Of course, the library file must be explicitly called by the config.js file, so as to be effectively loaded:

```
fn.getLibrary(sPathToLibrary, fnCallback, fnErrorhandler);
```

If you follow the rules hereabove, you won't need to declare any `sPathToLibrary`, because Lex'it will know where to look for it, so use the null value instead:

```
fn.getLibrary(null, fnCallback, fnErrorhandler);
```

The `fnCallback` argument is a function the code of which must/can be executed only after the library was loaded. Finally, you can add an error handler as a third argument, but this is null by default.

Note Loading multiple libraries at once is also possible, by using the `fn.getLibraries()` function (see section **Use multiple config.js files** for an example).

Use multiple config.js files

Building a complex project can result in a huge `config.js` file. To avoid that, you can spread the configuration across several files. To do so, first give each file the following structure:

```
// Just an example: in our project, we have a table called 'table_one'.
// Its configuration is declared in a file called 'my_project.table_1.js',
// containing the namespace 'table_1' (which holds the usual config parameters)

var table_1 = {};


// the content of the following is the same as any table settings declaration
// in oTableSettingsList

table_1.settings = {

	"width": "…", etc.
};

// the content of the following is the same as any table config declaration
// in oTableConfigurationList

table_1.config = {

	"row_id": {"…", etc.}
};
```

Each of those files can now be loaded as libraries. Put the following in your main `my_project.config.js`:

```
// declare the libraries to be loaded
var aLibrariesList = [
	"my_project.table_1.js",
	"my_project.table_2.js",
	"my_project.table_3.js",
	"my_project.somefunctions.js"
];


// function to set the usual oTableSettingsList and oTableConfigurationList
// like in any Lex'it project

function initializeTables(){

	oTableSettingsList = {
		"table_one": table_1.settings // declared in my_project.table_1.js
		"table_two": table_2.settings,// etc.
		. . .
	};
	oTableConfigurationList = {
		"table_one": table_1.config // declared in my_project.table_1.js
		"table_two": table_2.config,// etc.
		. . .
	};
};
```

As a last step, add the following code lines to start up the project:

```
// start up
fn.getLibraries(null, aLibrariesList, function(){

	// when all libraries are loaded, initialize!
	initializeTables();

	// here do anything you like
	...
});
```

By default, the `fn.getLibraries` function expects the library files to be found in the same folder as your main config file. If you with to put your library files in a subfolder instead, just replace the `null` value by the name this subfolder:

```
// start up
fn.getLibraries('my_libs_folder', aLibrariesList, function(){

	...
});
```

Load new CSS dynamically

Loading new CSS can be done in two ways: load a file, or set some CSS declaration straight away:

```
fn.getCssFile(sPath, fnCallback, fnErrorHandler);

fn.addCss("div#some_id {color: …, border: …}");
```

You can also set CSS variables, which can be used in the CSS styling, like this:

```
fn.setCssVariable("some_unit", "10px");

$(some_element).css("width", "calc("+factor+" * (var(--some_unit)))");
```

Accessing session and user info

Lex'it provides with some function for accessing session and user info. That allows for different operations, like logging who performed some data editing.

Logging who performed some data edit of a record could be implemented that way:

```
"some column": {
       "editable": true,
       "editcallback": function(t, n, value){
              var sRecordId = fn.getDataFromSiblingNode(n, "id");
              fn.insertIntoTable("log", {
                     "record_id": sRecordId,
                     "modified_by": fn.getCurrentUser(),
                     "modification_time": getCurrentTimestamp()
              });
       }
}
```

This solution is of course only recommended when the developer is not allowed to modify the (structure of the) project database. However, if you are entitled to modify the database, logging should be implemented there, so as to allow logging to occur at any occasion, even when the database wasn't processed by Lex'it:

```
CREATE TRIGGER log_trigger
    AFTER INSERT OR DELETE OR UPDATE ON yourtable
    FOR EACH ROW
    EXECUTE PROCEDURE do_some_logging();
CREATE FUNCTION do_some_logging()
    RETURNS trigger
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE
AS $BODY$
BEGIN
       INSERT INTO log(record_id, modified_by, modification_time)
       SELECT NEW.id, t.username, now()
       FROM active_user t;    -- this table is a temporary table
                              -- automatically created by Lex'it, which only exist
                              -- within the current session/transaction

       RETURN NULL; -- result is ignored since this is an AFTER trigger
END;
$BODY$;
```

Cleaning the cache

Lex'it caches data about tables. For example, counting the exact number of rows of a large table upon any request to do so, makes no sense. Once this number is known, it can be cached, and retrieved very quickly if requested once again.
This caching concerns mostly 'table statistics' like number of rows of the whole table, or of the result

set of a particular query, and 'table metadata', like primary keys, columns data types, etc. Note that the table content is *never* cached.

In some situation however, it might be needed to clean the cache, to make sure every single piece of information is reliable at that moment. To do so, call this:

```
cleanTableCache(t, fnCallBack, fnErrorHandler);
```

Setting behavior when switching between tabs containing distinct Lex'it instances

You can set a function to be executed when some Lex'it tab gains focus or loses it:

```
fn.doAtFocusGain( funtion(){

       fn.refreshTable(t);
});

fn.doAtFocusLoss( funtion(){

       ...
});
```

Changing accessed table schema

As explained in the first sections of this document, the `.database` table sets Lex'it to access only a particular schema of a database. It has the advantage of confining Lex'it to a particular area of the database, which allows for hiding less relevant schemata etc.

But in cases in which you need to switch to another database schema at runtime, call this function:

```
fn.setSchema(sNewSchema, fnCallback, fnErrorHandler);

// when run in the callback, this should yield a value matching sNewSchema:
var activeSchema = fn.getCurrentSchema();
```

Custom sorting

If some textual column has to be sorted in a special, custom way, several strategies are available.
An easy one is declaring a collation in the `.database` file of your project (see **My first project** section):

```
collation=nl_NL.utf8
```

However, if declaring a collation is not enough to produce the desired sort order, we'll have to assign a weight (rank) to each sign occurring in the column to be sorted, and put that in a table, like:

Sign	Rank
A	1
B	2
C	3
…	…

Once this is set, we must convert each cell of the textual column to be sorted into an integer array, in which each letter is replaced by its rank. The array must be stored in a new column named after the converted one with suffix '*_lexit_custom_sort*' added to the column name, like:

word	word_lexit_custom_sort
Abc	[1, 2, 3]
Book	[2, 15, 15, 11]
Think	[20, 8, 9, 14, 11]
…	…

When this naming convention is met, Lex'it will automatically recognize this new column and use it to sort the converted column according to the weights/ranks saved in the arrays.
If you wish to be able to sort such a column reversely as well, you must store the reversed arrays in another new column, this time with suffix '*_lexit_custom_reversesort*' added to the name of the converted column.

Here is an example for Russian:

```
CREATE TABLE public.russian_chars (
    "char" text COLLATE pg_catalog."en_US",
    rank integer DEFAULT 0
);

INSERT INTO public.russian_chars ("char", rank)
VALUES ('Б', 12), ('в', 13), ('В', 14), ('В', 15), ('в'', 16), ('Г', 18),
……, ('ѳ', 184), ('ѳ', 185);


-- split a Russian word into its separate chars
-- and deal with the fact that some chars are stored as TWO signs (char + accent)

CREATE OR REPLACE FUNCTION get_sep_russian_chars( input_word text )
    RETURNS text[]
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS
$BODY$
DECLARE
        word_arr integer[];
```

```
        output_arr text[];

BEGIN
        SELECT INTO word_arr array_agg( ascii(tekens) )
        FROM unnest( string_to_array($1, NULL) ) AS tekens;

        IF (array_lower(word_arr, 1) IS NOT NULL AND array_upper(word_arr, 1) IS NOT NULL)
THEN

                FOR i IN array_lower(word_arr, 1)..array_upper(word_arr, 1)
                LOOP
                  CONTINUE WHEN ( ARRAY[ word_arr[i] ] <@ ARRAY[46,8217,769,768,774,779] );

                  IF  ( i < array_upper(word_arr, 1) AND ARRAY[ word_arr[i+1] ] <@
ARRAY[46,8217,769,768,774,779] ) THEN
                        output_arr := ARRAY_APPEND( output_arr,
chr(word_arr[i])||chr(word_arr[i+1]) );
                  ELSE
                        output_arr := ARRAY_APPEND( output_arr, chr(word_arr[i]) );
                  END IF;
                END LOOP;
        END IF;

        RETURN output_arr;
END;
$BODY$;


-- Convert a russian word into an array, consisting of a sorting rank for each char of the
word

CREATE OR REPLACE FUNCTION convert_word_into_rank_arr(inputword text)
    RETURNS integer[]
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
DECLARE
        output_arr integer[];
BEGIN
        SELECT ARRAY(

                SELECT rc.rank
                FROM (
                        SELECT x.one_char, row_number() over () as row_nr
                        FROM (SELECT unnest(get_sep_russian_chars($1)) AS one_char )x
                ) chars
                LEFT JOIN russian_chars rc
                ON chars.one_char = rc.char
                WHERE rc.rank IS NOT NULL
                ORDER BY chars.row_nr
        ) INTO output_arr;

        RETURN output_arr;
END;
$BODY$;

-- now add the needed columns (properly named to allow lex’it to recognize those)

ALTER TABLE russian_table ADD COLUMN word_lexit_custom_sort integer[];
ALTER TABLE russian_table ADD COLUMN word_lexit_custom_reversesort integer[];

-- fill the columns with the generated arrays for custom sorting

UPDATE russian_table
SET word_lexit_custom_sort = convert_word_into_rank_arr(word);

UPDATE russian_table
SET word_lexit_custom_reversesort = (word_lexit_custom_sort);

-- and add indexes for performance

CREATE INDEX ON russian_table( word_lexit_custom_sort );
CREATE INDEX ON russian_table( word_lexit_custom_reversesort );
```

Create a Welcome or Login page

When a project is being accessed, Lex'it first shows a login dialog, requesting the user to enter a username and password.
In some projects, however, it might be more convenient to first display some introduction or instructions, before asking the user to log in. We call this a 'welcome page'.

Setting a welcome page is easy:
let's say your project is called '`my_project`'. Like in any Lex'it project, the functionalities of this project must be declared in a '`.config.js`' file, that is: '`my_project.config.js`'.
To set a welcome page, just create a file called '`my_project.welcome.js`', and put it in the same folder as the'`.config.js`' file.
Of course, the 'my_project' part of the '`my_project.welcome.js`' filename can be replaced by any name, be it must be the same name as in the '`my_project.config.js`' file. The fact that the project name matches in both filenames tells Lex'it that the two files belong together. That is: the '`my_project.welcome.js`' is the welcome page to be shown before loading '`my_project.config.js`'.

A far as the content of the welcome page file is concerned, only one thing is required: the file must call the login procedure programmatically. This is by design, as this allows you to decide how (and when) the login dialog must appear:

```
// call the login procedure
fn.startLexitLogin();
```

This code will call the login procedure straight away: a login dialog will appear on top of the welcome page content.
But you might want this dialog to appear only when clicking a login button:

```
$(some_element_of_the_page).append(
       $("<span></span>")
              .text( "Click here to log in" )
              .click(function(){
                     fn.startLexitLogin();
              })
);
```

Whatever choice you make, be sure to call the `fn.startLexitLogin()` at some moment, otherwise Lex'it will show the welcome page but will never open your project!

Call a Lex'it projects and tables via a URL

You can open Lex'it and go straight to a given project (that is: skipping the menu page) by specifying the project name in the URL:

```
http://<your_host>:8080/lexit2/?db=myproject
```

It is even possible to open a project and call a table straight away. To do so, specify the table name too:

```
http://<your_host>:8080/lexit2/?db=myproject&table=sometable
```

The next step is to get the table filtered by some values right from the start as well. Once again: just specify the filters to be applied!

```
http://<your_host>:8080/lexit2/?db=myproject&table=sometable
    &columnName1=someValue1&columnName2=someValue2
```

Instead of targeting specific columns to be searched, you can just search the whole table at once:

```
http://<your_host>:8080/lexit2/?db=myproject&table=sometable
    &anycolumn=someValue
```

Finally, you can also set some particular settings, like the table screen position or so. Settings can be specified the same way as anything else, but the setting name must have the 'setting.' prefix, like this:

```
http://<your_host>:8080/lexit2/?db=myproject&table=sometable
    &setting.top=150px&setting.left=20px
```

Note: a list of all available settings cans be found in section **First things first: open a table**.

Set the language of the GUI

By default, all textual messages in Lex'it are put in Dutch. But it is possible to have the software speak another language.
Let's say you expect Lex'it to communicate in English. That can be achieved in two ways. The first is by adding the '`lang`' parameter to the URL called to start Lex'it:

```
http://<your_host>:8080/lexit2/?db=myproject&lang=en
```

Obviously, the '`en`' value codes for the English language. The second way to tell Lex'it to speak English is by setting it programmatically in the project code:

```
lang.setLanguage("en");
```

Change table columns visibility and redraw table automatically

Once a table is rendered on screen, it is still possible to modify the columns' visibility programmatically, so as to switch between alternative views of the data.
Declare the list of columns (`aListOfColumns`) that have to be visible, and call the following function:

```
fn.changeTableColumnsVisibility(t, aListOfColumns, fnCallback, oExtraSettings);
```

This function takes care of redrawing the table properly at the very same screen position, and re-applies filters, view type, pagination, etc. If needed, some extra custom settings can be passed on with argument `oExtraSettings` (see section **The API, First things first: open a table**) and some callback can be applied to with `fnCallback`.

Scroll to a table programmatically

When a table is opened programmatically, and other tables are already open, the newly opened table might not be visible as it is automatically put at the bottom of all others. In such cases, it might be convenient to automatically scroll to it:

```
fn.scrollToTable(sTableName);
```

Synchronicity problems and solutions

When a function calls a webservice or some database function etc., we need the function that will process the results to be waiting till those are issued. Otherwise we'll end up with undefined variables etc.

In Lex'it, this is supported by the implementation of callbacks. Most of the function allow one or more callbacks as a parameter. It's only when the function execution is finished, that the callback will be run:

```
fn.updateTableGivenFieldValues( t,
      {"column1": someValue1},
      {"column2": someValue2, "column3": someValue3},
      ...,
      function(){  // put some nice callback code here });
```

Sometimes you might want to execute a chain of operations, in such a way that each operation waits for the previous one to be finished before starting the current one, and so on. That can be achieved in different ways: with recursion, or by using queues..

Let's start with recursion. In the following example, we gather the IDs of some rows, onto which some operations will be carried out. As a second step, we define a function to be executed on one row at the time. Since the function can process only one row at once, we want it to process the following row only when it's done with the current one.
To do that, we have the function call itself in a callback. The parameter value is increased at each call, ensuring that the next ID of the list is chosen for processing. The recursion ends when the last ID was reached.

The whole process is started by calling the function with parameter value 0, telling the function to pick up the very first element of our list of IDs. It will end when every ID is processed:

```
var aIdsToProcess = new Array();
var aSelectedAtts = fn.getSelectedRowsFrom(t);
aSelectedAtts.each(function(){
    aIdsToProcess.push( fn.getNodeId(this) );
});

// process each selected row now
var functionToRepeat = function(i){
    fn.updateTableGivenFieldValues( t,
            {"id_column": aIdsToProcess[i]},
            {"column1": someValue1, "column2": someValue2},
            false,
            function(){
                if ( (i+1) < aIdsToProcess.length ) {
                    functionToRepeat(i+1);
                }
                else {
                    // finished? Refresh to see the results!
                    fn.refreshTable(t);
                }
            });
};

// start the function now
// it will increment its own argument till all ids are processed
functionToRepeat(0);
```

Another way of dealing with synchronicity issues is working with queues. That's easy: just declare a queue, and push the functions to be executed to it. Each function from the queue will be automatically executed, in the same order it was pushed to the queue:

```
const queue = new FunctionQueue();

queue.enQueue(function(){ ... }); // task 1
queue.enQueue(function(){ ... }); // task 2
```

Implement a radio button using Boolean columns

Let's say we have three Boolean columns designed to hold the gender of a lemma: 'male', 'female' or 'neuter'. And only one at the time can be true: if male is true, female and neuter must be false, etc. We can implement that the following way:

```
// declare the fields that have to behave as a set radio buttons
var editionTypeFields = ["male", "female", "neuter"];


// declare a callback, setting the radio button behaviour
var fnRadioCallback = function(n, fieldsToUpdate, fieldName) {

       // set the other fields to false
       var updatedFields = {}
       fieldsToUpdate.filter(f => f != fieldName).forEach(f => updatedFields[f] =
false );

       var nRow = fn.getRowNode(n);
       fn.updateTableGivenANode(nRow, updatedFields, function(){
fn.callRecord(nRow, fieldsToUpdate) });
      }

  return editcallbackfunction
}

oTableConfigurationList = {

// usual table declaration, implementing the callback declare hereabove

   some_table: {
       "male": {
              "editable": true,
              "editcallback": function(t, n, value){
                     fnRadioCallback(n, editionTypeFields, "male");
              }
       },

       "female": {
              "editable": true,
              "editcallback": function(t, n, value){
                     fnRadioCallback(n, editionTypeFields, "female");
              }
       },
       "neuter": {
              "editable": true,
              "editcallback": function(t, n, value){
                     fnRadioCallback(n, editionTypeFields, "neuter");
              }
       }
   }
}
```

Implement a character picker

When you need to type in data consisting of foreign characters, with diacritics, anyhow characters not available on a current keyboard, a character picker can be the right solution. All you need is a table (let's call it 'character_picker') with one single cell, in which all needed characters are displayed, separated by spaces. Picking up a character can be done by just clicking on it.

```
"character_picker": {

            "character_picker_cell": {

                  "textsize": "20pt",
                  "click": function( t, n ){

                        // copy chars clicked upon
                        var oCell = fx.getCell(n, " character_picker_cell ");
                        var oClickedUpon = fx.getWordClickedUponInCell(oCell);
                        var sClickedUpon = oClickedUpon.text;

                        // put the chars into an invisible textarea
                        var tmpTextArea = $("<textarea></textarea>")
                        .attr("id", "chosen_character").text(sClickedUpon);
                        $("#temporary_stuff").append(tmpTextArea);

                        // and select it!
                        var copyChar = $('#chosen_character');
                        copyChar.select();

                        try {
                              // now copy it to clipboard
                              document.execCommand('copy');

                              // confirm to user which chars he/she has chosen
                              fn.message("Chosen character",
                              "Chosen character:<BR><BR>"+sClickedUpon);

                              // remove temporary textarea
                              $("#chosen_character").remove();
                              setTimeout(function(){

                                    // remove message
                                    fn.closeDialog();

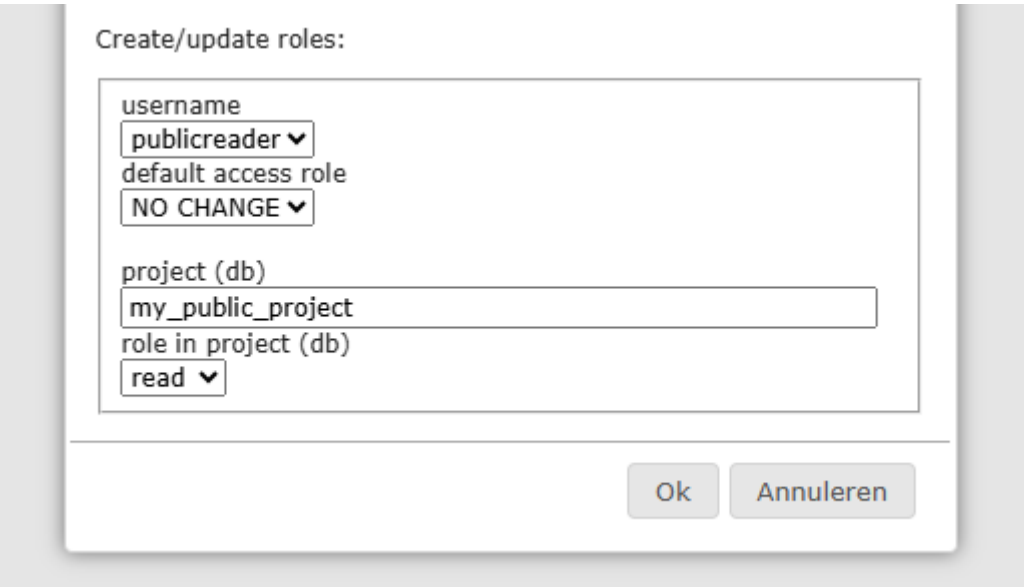
                                    // put focus onto the main table
                                    // this will call a header function,
                                    which will make this table active
                                    $("#mytable_wrapper div.top").mousedown();
                                    }, 1000);
                                } catch (err) {

                                // tell user if something went wrong
                                fn.message("Selection failed, try again!");
                                }
                        }
                  }
            }
```

Allow users in without logging in

If you wish to publish some data on the internet using the Lex'it GUI, you'll probably want to grant free access to users, that is: access without needing to enter any password.

To achieve that, you need to set the 'publicreader' of Lex'it. This is a default user, with no rights at all. So, first get to the admin GUI (see section **Assign rights to users** in this document), choose the 'publicreader' user, and assign it the 'read' role for the project you wish to make publicly available:



Note: by default, the 'publicreader' has no default access role and that can't be changed, which is by design, as we wish to prevent this user from gaining too much access rights. For the same reason, you can only assign read access: any attempt to assign more rights will fail.

When the above is done, you'll have to tell the Lex'it project to log in automatically as 'publicreader'. To do so, create a welcome page for your project (see section **Create a Welcome or Login page** in this document) and add this single line to it:

```
fn.startPublicReader();
```

From now on, anyone can access your project without having to enter a password. Of course, visitors won't be able to edit the data, but they have full reading access.

Help functions

Lex'it has several utility functions, dealing with other objects than tables. Some are part of the `fn` namespace, other are included in a `lexutil` namespace (check this namespace since some functions might be missing in this document):

String array conversion

Convert an array to a string, or back:

```
var array = ["app", "noot", "mies"];
var str = fn.arrayToString(array); // outputs the array as a string

var string = "{\"app\", \"noot\", \"mies\"};"
var arr = fn.stringToArray(str);
```

Escaping chars

Escape quotes in string parameters of function or service call:

```
fn.escapeSingleQuotes(str);

fn.escapeDoubleQuotes(str);
```

Escape regular expression characters, for example for using some string in a search query:

```
var sSearchString = fn.escapeRegexChars( someStringContainingRegexChars );

fn.callTable(t, {"some_column": sSearchString});
```

Http parameters

Get the http parameters of Lex'it (like value of params `db`, `test`, etc.):

```
var hParams = lexutil.getHttpParams(); // this is a Hashtable
var sDb = hParams.get("db");
```

Solve z-Index problems

Put any object in front (solves many z-index problems):

```
$(obj).putInFront();
```

Objects functions

Clone an object

```
var oClone = NEW lexutil.cloneObject(obj);
```

Count the number of properties of an object

```
var iCount = lexutil.countProperties(obj);
```

Regex functions

Test if a string is a regex or not

```
var bIsRegex = lexutil.isRegex( str );
```

Regex version of replaceAll(string, replacement)

Note: the search argument must be a string, and not a regex literal, that is: `"^(...)$"` instead of `/^(...)$/` :

```
var sNewString = str.regexReplaceAll( regex, replacement );
```

Regex version of indexOf(regex, from) and lastindexOf(regex, from)

```
var firstMatch = str.regexIndexOf( regex, fromStartPos );
var lastMatch =  str.regexLastIndexOf( regex, fromStartPos );
```

Regex version of inArray(value, array, start)

```
var idx = $.inArrayRegEx( value, array, start );
```

Regex selectors

Lex'it supports jQuery selector defined as regular expressions.
Examples of use in Lex'it implementation:

```
$(':regex(id,^[aeiou])');
$('div:regex(class,[0-9])');
$('script:regex(src,jQuery)');
```

Arrays functions

Unify any value in array

```
var aNoDoubles = lexutil.getOnlyUniqueValues( arr );
```

Clone an array

```
var aClone = lexutil.cloneArray( arr );
```

Array equality

Check for array equality: this means same length, same order and same elements values.

```
var bEqual = lexutil.arraysAreEqual( arr1, arr2);
```

Sort an array of arrays

```
var aaSorted = lexutil.sortArrayOfArray( aaArray );
```

HTML entities

Test if a given HTML entity is a common (internationally known) entity:

```
var bTrueOrFalse = lexutil.isCommonEntity( sSomeEntity );
```

Translate an entity into the character it represents:

```
var sChar = lexutil.translateEntityToChar( sEntity );
```

Translate a character into the corresponding entity:

```
var sEntity = lexutil.translateCharToEntity( sChar );
```

Numbers

Generate series (JavaScript version of PostgreSQL function)

Note: the generated series contains leading zeros (like: `002`, `003`, …) so as to match the length of the biggest member of the series (`iEndValue`).

```
var aStr = lexutil.generateSeries( iStartValue, iEndValue, bAscending );
```

Get a random unique number

Based on the current date, this number is surely unique:

```
var iUniqueNumber = lexutil.getUniqueNumber();
```

String functions

The good old BASIC or JAVA functions

```
var sLeftPart =  lexutil.left( str, n );
var sRightPart = lexutil.right( str, n );

var bTrueOfFalse = $.startsWith( str, search );
var bTrueOfFalse = $.endsWith( str, search );
```

String replacement

Normal and regex version:

```
var sNewStr = str.replaceAll( regex, replacement );
var sNewStr = str.regexReplaceAll( regex, replacement );
```

Check for HTML tags, and remove those from a string

```
if (lexutil.hasTags( str ) {
       var sCleanStr = lexutil.removeTags( str );
}
```

Remove non alphabetical chars from string

```
var sCleanStr = lexutil.keepOnlyLettersAndDigits( str );
```

If a string a string?

```
var bTrueOfFalse = $.isString( str );
```

If a string truly empty?

Check if a string is null, undefined or just a zero-length string:

```
var bTrueOfFalse = $.emptyString( str );
```

Colors

Is a color dark or light?

```
// parameter might be RGB or HEX color code
var lightOrDark = lexutil.lightOrDark( color );
```

Convert a string to a color

Note: sometimes convenient for random color generation:

```
// output is HEX color code
var sColor = lexutil.stringToColor( str );
```

Generate some color code, but make sure it's not too light or so:

```
// generate random color for group of variants
sColor = lexutil.stringToColor( sLem );
while(lexutil.lightOrDark(sColor) == 'light'){
       sColor = lexutil.colorLuminance( sColor, -0.1 )
}
```

Make color a bit lighter or darker

```
// the second parameter must be positive for lighter, negative for darker color
var newColor = lexutil.shadeColor("#AA2222", -10);
```

Convert HEX to RGB color code, or back

```
var sHex = lexutil.rgbToHex(r, g, b);
var aRgb = lexutil.hexToRgb(sHex); // returns {r:..., g:..., b:...}
```

Text selection

Allow or prevent screen text selection

```
$('html, body').enableSelection();

$('html, body').disableSelection();
```

Clear screen text selection

Prevent the browser from selecting the whole page upon clicking a button or so:

```
lexutil.clearSelection();
```

DOM elements functions

Does an element exist?

```
var bElementIsThere = $(el).elementExists();
```

Test if some element is visible in the current viewport

```
var bVisible = $(el).isOnScreen(); // true if any portion of the element is
visible
```

Events

Pre-binding

Pre-bind a function, that is: bind a handler to some event, to be executed before all other handlers of the same event:

```
$('#button').preBind('click', function() {
        console.log('hello');
});
```

PSQL functions in Lex'it

Call a PSQL-function and read the result

A PSQL function (say `api.do_something`) can be called in Lex'it in the following way:

```
fn.callFunction("api.do_something", [arg1, arg2],

       function(resp){ … }
);
```

The output (`resp`) of this Lex'it function is always an associative array. However, the way its output must be read, depends on the output type of PSQL function being called:

If the PSQL function returns a **simplex** result like an single string or so (… `RETURNS text` …), the output will be associated to a key named after the PSQL function:

```
function(resp){

       var ourResult = resp["do_something"];

       doWhatEver( ourResult );
}
```

As you probably already noticed, the schema name (`api.`) mustn't be used in the key name when retrieving the output. Only the function name (`do_something`) is to be used as a key to the output value.

But if the function returns a **complex** result type, like a table record (… `RETURNS TABLE(id integer, some_string text)` …), the value of each table column will be associated to a key named after the columns name:

```
function(resp){

       var ourId =     resp["id"];
       var ourString = resp["some_string"];

       doSomething( ourId, ourString );

}
```

Concordance table for the fn and fx namespaces

fn functions	fx functions
fn.addCustomButton(…)	
fn.addDrawCallback(…)	
fn.addFilters(…)	
fn.alignTables(…)	
fn.arrayToString(…)	
fn.askToChoose(…)	
fn.callTable(…)	
fn.callTableInNewTab(…)	
fn.callFunction(…)	
fn.callRecord(…)	fx.callRecord(…)
fn.callService(…)	
fn.changeTableColumnsVisibility(…)	
fn.cleanTableCache(…)	
fn.clearAllFilters(…)	
fn.clickOnButton(…)	
fn.closeDialog()	
fn.closeTable(…)	
fn.closeAllTables(…)	
fn.confirm(sTitle, …)	
fn.declareNewTable(…)	
fn.doAtFocusGain(…)	
fn.duplicateRecord(…)	
fn.editFunctionIsActiveNow()	
fn.escapeDoubleQuotes(…)	
fn.escapeRegexChars(…)	
fn.escapeSingleQuotes(…)	
fn.forceExactCount()	
fn.getActiveRowNode(…)	fx.getActiveRow(…)
fn.getAllowedValuesOfColumn(…)	
fn.getAllRowNodes(…)	fx.getAllRows(…)
fn.getAllRowNodesWhere(…)	fx.getAllRowsWhere(…)
fn.getCellInRowNode(…)	fx.getCell(…)
	fx.getCellNode(…)
fn.getCellNodeType(…)	fx.getCellType(…)
fn.getCheckboxValue(…)	fx.getCheckboxValue(…)
fn.getColumnNumberOf(…)	
fn.getCurrentDisplayLength(…)	
fn.getCurrentDisplayStart(…)	
fn.getCurrentPageIndex(…)	
fn.getCurrentProject()	
fn.getCurrentSessionId()	
fn.getCurrentTabId()	
fn.getCurrentTimestamp(…)	
fn.getCurrentUser()	
fn.getCustomButtonCss(…)	
fn.getDataFromCellNode(…)	fx.getDataFromCell(…)

fn.getDataFromCellInRowNode(…)	fx.getDataFromCellInRow(…)
fn.getDataFromColumn(…)	
fn.getDataFromRowNode(…)	fx.getDataFromRow(…)
fn.getDataFromSiblingNode(…)	fx.getDataFromSiblingCell(…)
fn.getFilters(…)	
fn.getFirstRowNodeFrom(…)	fx.getFirstRowFrom(…)
fn.getFirstSelectedRowNodeFrom(…)	fx.getFirstSelectedRowFrom(…)
fn.getFunctionOutput()	
fn.getGlobalFilter(…)	
fn.getHighlight(…)	
fn.getIdFromTable(…)	
fn.getIndexOfButtonNamed(…)	
fn.getIndexOfFirstSelectedRowNodeFrom(…)	fx.getIndexOfFirstSelectedRowFrom(…)
fn.getLibrary(…)	
fn.getListOfColumnsNiceNames(…)	
fn.getNameOfButtonAtIndex(…)	
fn.getNameOfColumnForThisNode(…)	fx.getNameOfColumnForThisCell(…)
fn.getNameOfColumnForThisVisibleIndex(…)	
fn.getNewPositionOfElementAt(…)	
fn.getNextRowNode(…)	
	fx.getNode(…)
fn.getNumberOfSelectedRowNodes(…)	fx.getNumberOfSelectedRows(…)
fn.getNumberOfVisibleRows(…)	fx.getNumberOfVisibleRows(…)
fn.getPreviousRowNode(…)	
fn.getPromptBoxInput(…)	
fn.getPromptBoxOrder()	
fn.getRecord(…)	
fn.getRecordGivenFieldValues(…)	
fn.getRecords(…)	
fn.getRowNode(…)	fx.getRow(…)
fn.getRowNode(…)	fx.getRowFromCell(…)
fn.getRowNodeId(…)	fx.getRowId(…)
fn.getRowNodeIndex(…)	fx.getRowIndex(…)
fn.getRowNodeNumberOnScreen(…)	fx.getRowNumberOnScreen(…)
fn.getRowNodeWhere(…)	fx.getRowWhere(…)
fn.getRowNodeWhereIdIs(…)	fx.getRowWhereIdIs(…)
fn.getSelectedRowNodesFrom(…)	fx.getSelectedRowsFrom(…)
fn.getSelectedTextInNode(…)	fx.getSelectedTextInCell(…)
fn.getSortingColumns(…)	
fn.getSortingDirection(…)	
fn.getSortingDirectionOf(…)	
fn.getSortingDirections(…)	
fn.getTableExtraSettings(…)	
fn.getTableNode(…)	fx.getTableNode(…)
fn.getTableName(…)	fx.getTableName(…)
fn.getTablePosition(…)	
fn.getTotalNumberOfPages(…)	
fn.getTotalNumberOfRows(…)	
fn.getTypeOfFilterBox(…)	
fn.getValueOfFilterBox(…)	

fn.getViewType(…)	
fn.getVisibleColumnNumberOf(…)	
fn.getWordClickedUponInNode(….)	fx.getWordClickedUponInCell(…)
fn.goToNextPage(…)	
fn.goToPreviousPage(…)	
fn.goToTheRightPage(…)	
fn.hideTable(…)	
fn.insertIntoTable(…)	
	fx.isApiInstance(…)
fn.isCellNode(…)	fx.isCell(…)
fn.isEditableNode(…)	fx.isEditableCell(…)
fn.isLastNodeOf(…)	fx.isLastRowOf(...)
fn.isRowNode(…)	fx.isRow(…)
fn.message(…)	
fn.pileupTables(…)	
fn.pressKey(…)	
fn.preventAutomaticStartup()	
fn.processPromptBoxOrder(…)	
fn.prompt(…)	
fn.promptReorder(…)	
fn.promptSelect(…)	
fn.putDataIntoCellNode(…)	
fn.putDataIntoFilterBox(…)	
fn.putDataIntoCellNode(…)	fx.putDataIntoCell(…)
fn.quote(…)	
fn.refreshTable(…)	
fn.refreshMaterializedView(…)	
fn.registerNewTable(…)	
fn.removeFromTableGivenANode(…)	fx.removeFromTableGivenARow(…)
fn.removeFromTableGivenFieldValues(…)	
fn.removeHighlight(…)	
fn.removeProcessingMsg(…)	
fn.resetAllFilters(…)	
fn.resetTable(…)	
fn.restoreTableState(…)	
fn.rowsSelectionIsAllowed(…)	
fn.saveTableState(…)	
fn.scrollToTable(…)	
fn.selectAllRowNodes(…)	fx.selectAllRows(…)
fn.selectRowNode(…)	fx.selectRow(…)
fn.setActiveTable(…)	
fn.setAutoComplete(…)	
fn.setBackgroundColor(…)	
fn.setCssVariable(…)	
fn.setCurrentUser(…)	
fn.setCustomButtonCss(…)	
fn.setCustomButtonName(…)	
fn.getCurrentSchema()	
fn.setFilters(…)	
fn.setGlobalFilter(…)	

fn.setProjectTitle(…)	
fn.setSchema(…)	
fn.setSorting(…)	
fn.setTableNameInHeader(…)	
fn.showProcessingMsg(…)	
fn.showTable(…)	
fn.showTabs(…)	
fn.stringToArray(…)	
fn.tableExists(…)	
fn.tableIsOpen(…)	
fn.tableIsEmpty(…)	
fn.tableIsHidden(…)	
fn.toggleCheckbox(…)	fx.toggleCheckbox(oRow, …)
fn.triggerKeyStrikeOnElement(…)	
fn.uncheckCheckboxes(…)	fx.uncheckCheckboxes(…)
fn.unselectAllRowNodes(…)	fx.unselectAllRows(…)
fn.unselectRowNode(…)	fx.unselectRow(…)
fn.updateTableGivenANode(…)	fx.updateTableGivenACellOrRow(…)
fn.updateTableGivenFieldValues(…)	

Concordance table for forms vs. other namespaces

forms functions	other namespaces
form.resetSearchFields(…)	fn.clearAllFilters(…)
form.getColumnConfig(…)	conf.getColumnConfig(…)
form.getColumnsNiceNames(…)	conf.getNiceName(…)
form.getDataFromCell(…)	fn.getDataFromCellNode(…)
form.getFeedingTable(…)	fn.getTableName(…)
form.getNameOfCell(…)	fn.getNameOfColumnForThisNode(…) fn.getNameOfColumnForThisVisibleIndex(…)
form.getVisibleColumns(…)	fn.getVisibleColumns(…)
form.isUnsaved()	
form.refresh(…)	fn.refreshTable(…)
form.resetForm(…)	fn.refreshTable(…)
form.resetSearchFields(…)	fn.resetAllFilters(…)
form.saveForm(…)	
form.setDataInCell(…)	fn.putDataIntoCellNode(…)
form.updateLayout(…)	fn.refreshTable(…)

Concordance table for lists vs. other namespaces

lists functions	other namespaces
lists.addRowToList(…)	fn.insertIntoTable(…)
lists.clickOpenInSelectedRow(…)	
lists.getAllColumns(…)	fn.getAllColumns(…)
lists.getAllowedValuesOfColumn(…)	fn.getAllowedValuesOfColumn(…)
lists.getCell(…)	fn.getCellInRowNode(…) fx.getCellNode(…)
lists.getCellConfig(…)	conf.getColumnConfig(…)

lists.getColumnsToUse(…)	
lists.getColumnType(…)	fn.getCellNodeType(…)
lists.getColumnsToDisplay(…)	fn.getVisibleColumns(…)
lists.getColumnsWidth(…)	conf.getWidthSetting(…)
lists.getConfig(…)	conf.getTableConfig(…)
lists.getContainerIdFromNode(…)	
lists.getDataFromCell(…)	fn.getDataFromCellNode(…)
lists.getDataFromColumn(…)	fn.getDataFromCellInRowNode(…)
lists.getDataTableObjectOf(…)	mt.getDataTableObjectOf(…)
lists.getDisplayHeight(…)	
lists.getFeedingTable(…)	fn.getTableName(…)
lists.getLabelFromContainerId(…)	
lists.getLabelFromDataTableObject(…)	
lists.getLabelFromNode(…)	fn.getTableName(…)
lists.getListOfListLabels(…)	mt.getListOfLoadedTables()
lists.getNiceColumnName(…)	conf.getNiceName(…)
lists.getNumberOfVisibleRows(…)	fn.getNumberOfVisibleRows(…)
lists.getRow(…)	fn.getRowNode(…) fn.getRowNodeWhere(…)
lists.getSelectedRows(…)	fn.getSelectedRowNodesFrom(…) fx.getSelectedRowsFrom(…)
lists.getSiblingCell(…)	fn.getCellInRowNode(…)
lists.getSortSettings(…)	conf.getDefaultSortingFromTableSettings()
lists.getTableIdFromNode(…)	
lists.getTableIdFromContainerId(…)	
lists.getTableIdFromDataTableObject(…)	
lists.getTrueColumnName(…)	mt.getListOfColumnsOf(…)
lists.getVisibleColumns(…)	fn.getVisibleColumns(…)
lists.refresh(…)	fn.refreshTable(…)
lists.removeProcessingMsg(…)	fn.removeProcessingMsg(…)
lists.removeRowFromList(…)	fn.removeFromTableGivenANode(…) fx.removeFromTableGivenARow(…)
lists.selectRow(…)	fn.selectRowNode(…) fx.selectRow(…)
lists.setDataInCell(…)	fn.putDataIntoCellNode(…)
lists.showProcessingMsg(…)	fn.showProcessingMsg(…)
lists.unselectRow(…)	fn.unselectRowNode(…) fx.unselectRow(…)